Technische Universität Berlin

Distributed Systems
Chapter 4: Programming Models

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

4.1 Fundamental Programming Model

4.2 Service Oriented Architectures

4.3 Event Driven Architecture

4.4 Agentic AI

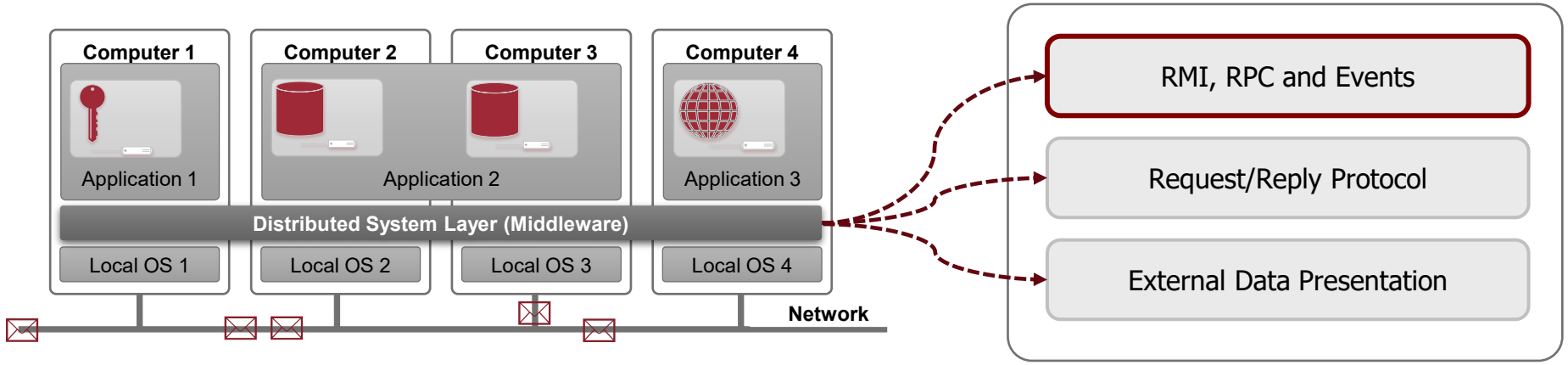
4.5 Enterprise Application Integration

Programming Models

4.1 Fundamental Programming Model

Fundamental Programming Model

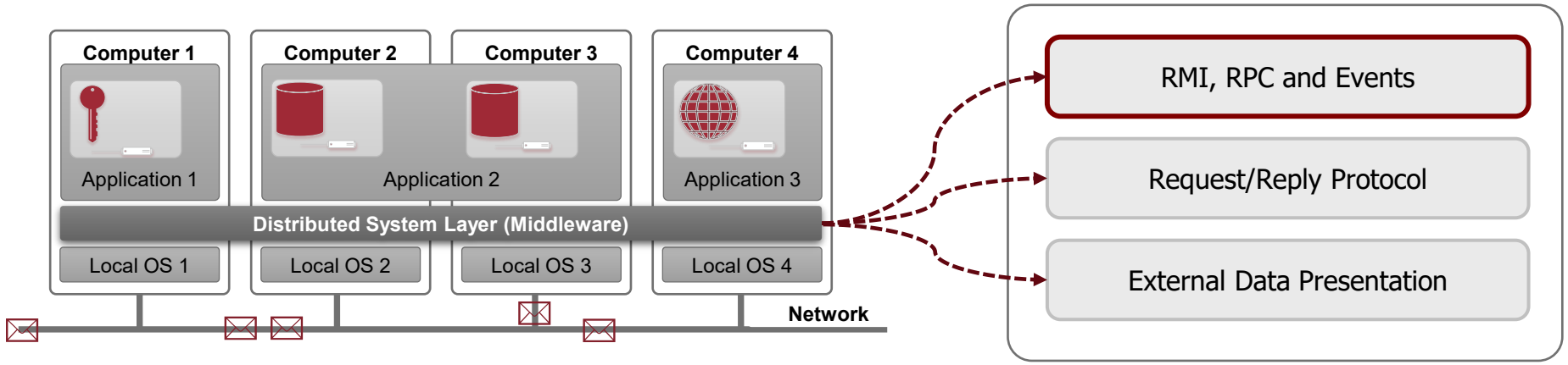
Distributed applications consist of components which are executed in different processes and cooperate



Fundamental Programming Model

Basics: Remote method invocation

- Distant processes provide methods over a remote interface
- In contrast to local invocations: Several failure sources are possible → total transparency (i.e. identical behavior as with local invocations) is not desirable



Fundamental Programming Model

RPC-Model: invocations of procedures and their execution with corresponding parameters in disjunct address spaces

Object-based approaches: communication between objects in different processes and invocation of methods in distant objects

Component-based approaches: composition of several objects in a higher level of abstraction (e.g. container)

Event-based programming: subscription of objects to events that occur on remote objects and respective notification

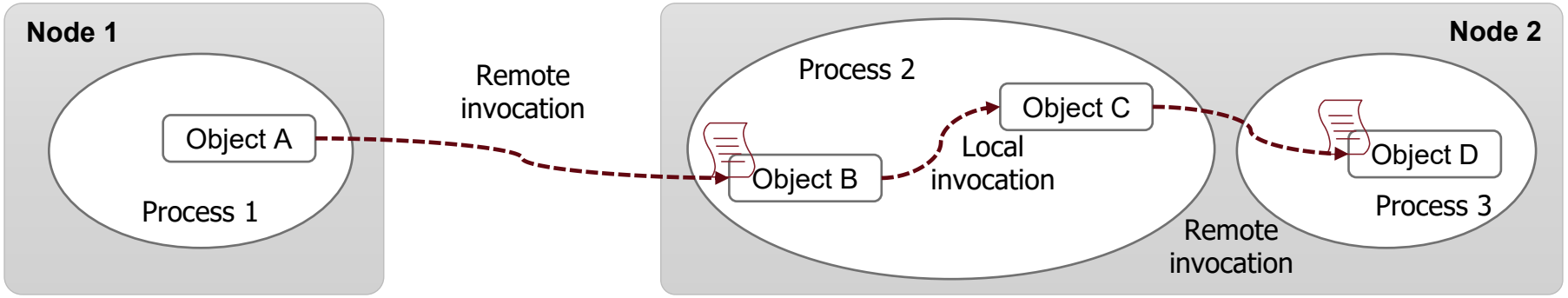
Web-Services, Representational State Transfer (REST): Clients can apply CRUD (create, read, update, delete) operations on resources

Agentic AI: autonomous agents implementing user intentions

Distributed Object Model

Extension of object model, enabling both local and remote access to objects

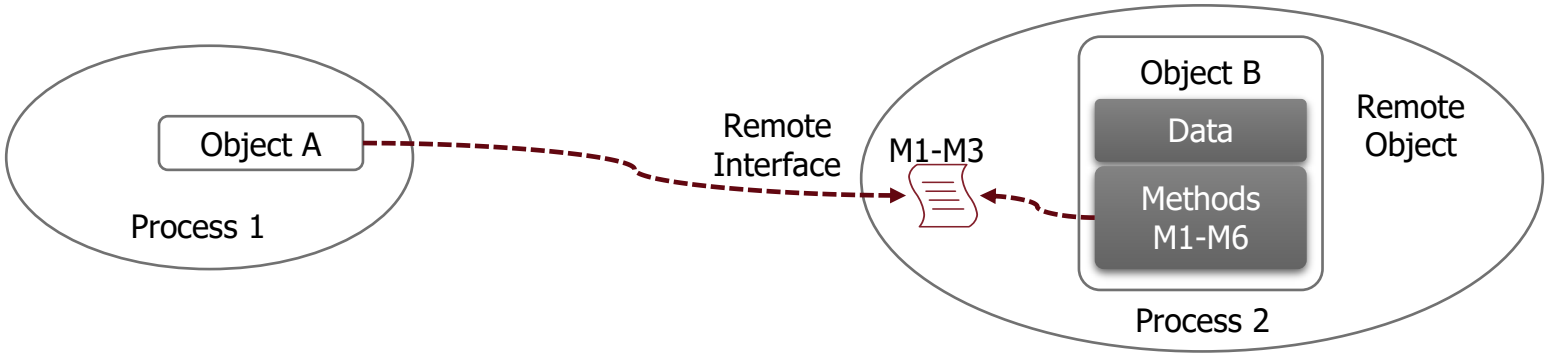
- Local method invocations between objects in the same process
- Method invocations between objects in remote processes → Remote Method Invocation (RMI)
  - Objects able to receive remote invocations are called external or remote objects (objects B and D)
  - Objects can receive local invocations from those objects that have references to this object



Distributed Object Model

Two key components of the Distributed Object Model

1. Remote Object Reference: Other objects can invoke the methods of a remote object if they have access to its remote object reference
2. Remote Interface: determines which methods of the external objects are allowed to be invoked



Distributed Object Model

Every object that can receive remote calls must have a remote object reference

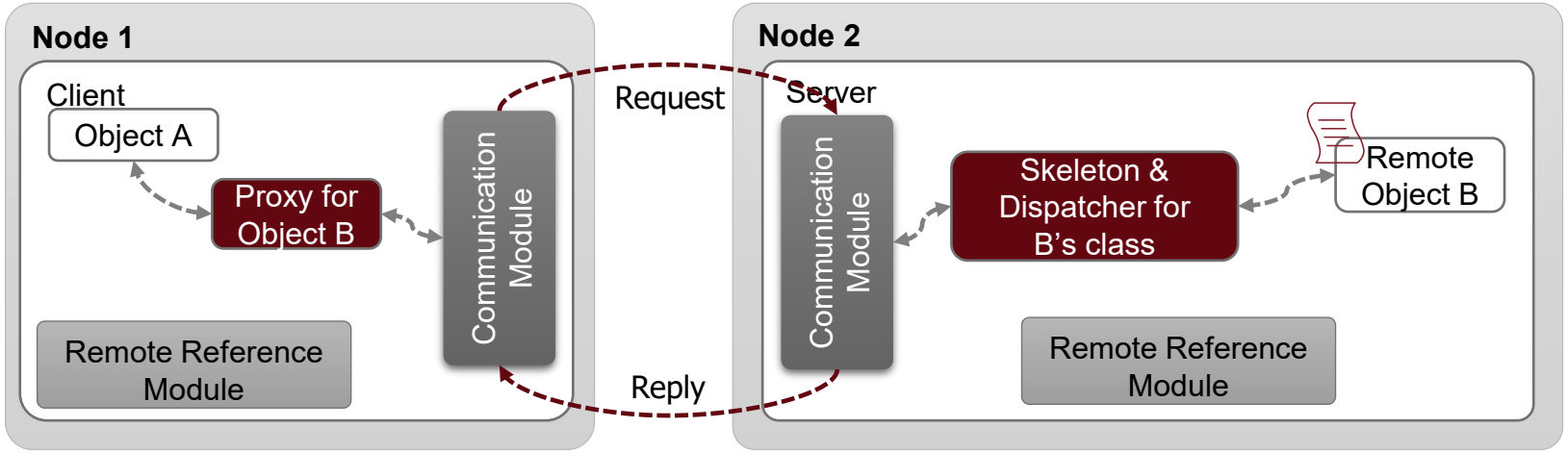
- Globally unique identifier that is used as a reference to a remote object
- Remains unique over time and space
- Transferred in the form of arguments and results of remote method invocations

Exemplary composition of a remote object reference:

- Interface: describes the remote interface of an object
- Protocol, address: protocol, host-domain-name and port number distinctly identify the process
- Object key: identifies object to be invoked

Distributed Object Model – Implementation Aspects

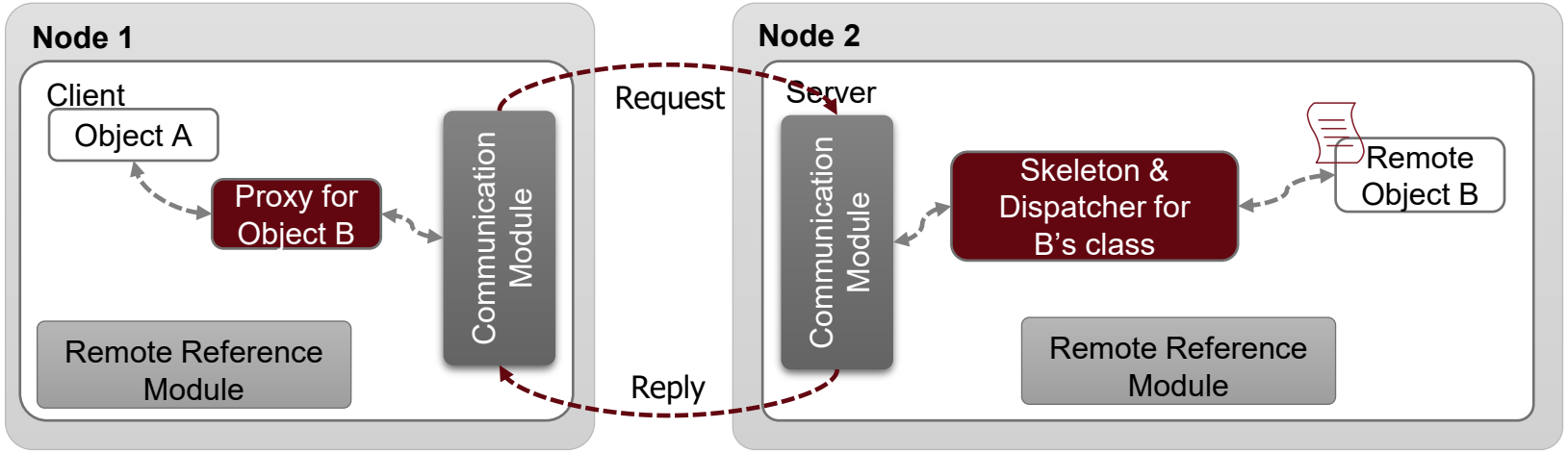
Communication module: Cooperate with request/reply protocol



Distributed Object Model – Implementation Aspects

Remote reference module:

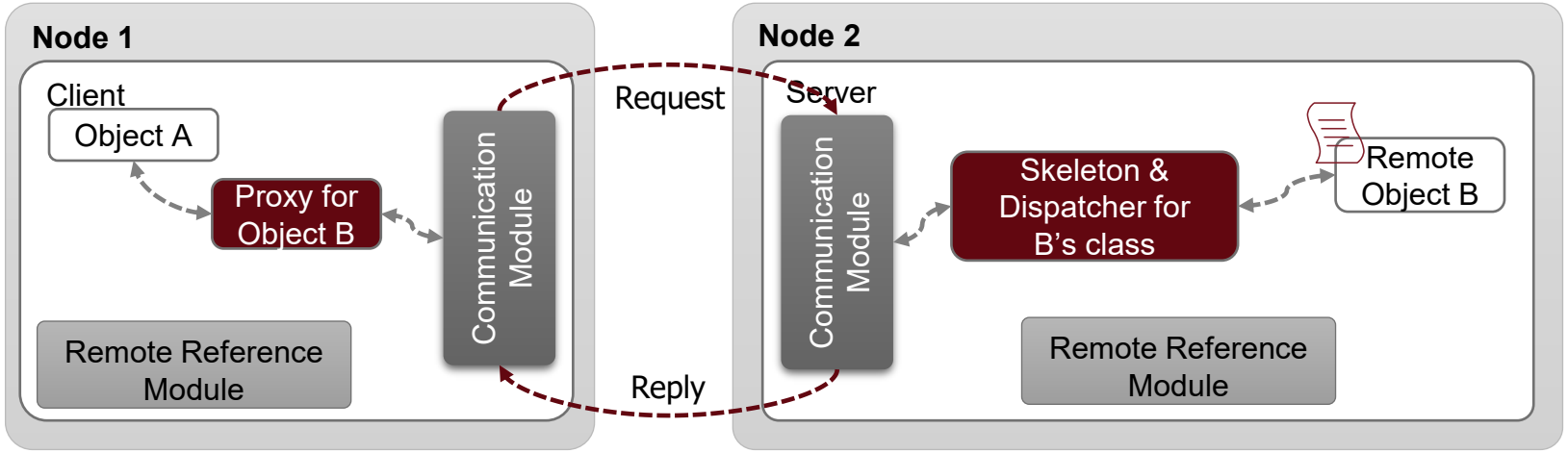
- Translates between local/remote references with the help of an object table
- At arrival of remote object reference: selects corresponding local object reference
- First invocation: creates new proxy (Node 1) and adds it to the object table (Node 2)



Distributed Object Model – Implementation Aspects

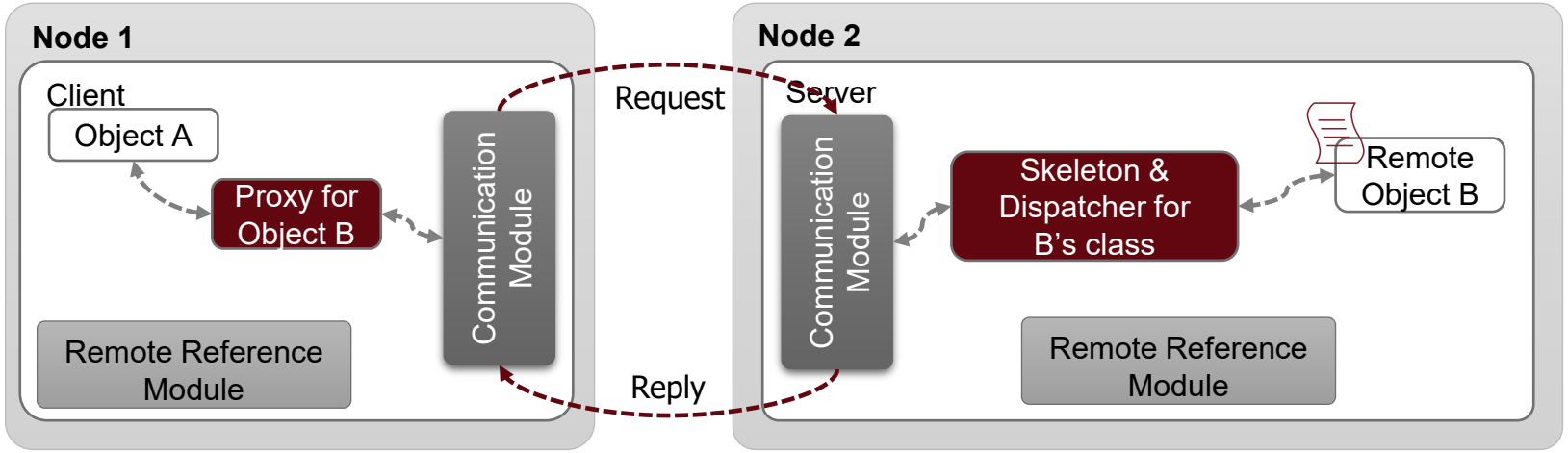
Proxy (delegate): makes remote access, (un-)marshalling and transmission transparent to clients

- Proxy has methods for parameter check and enhancing messages with references and data



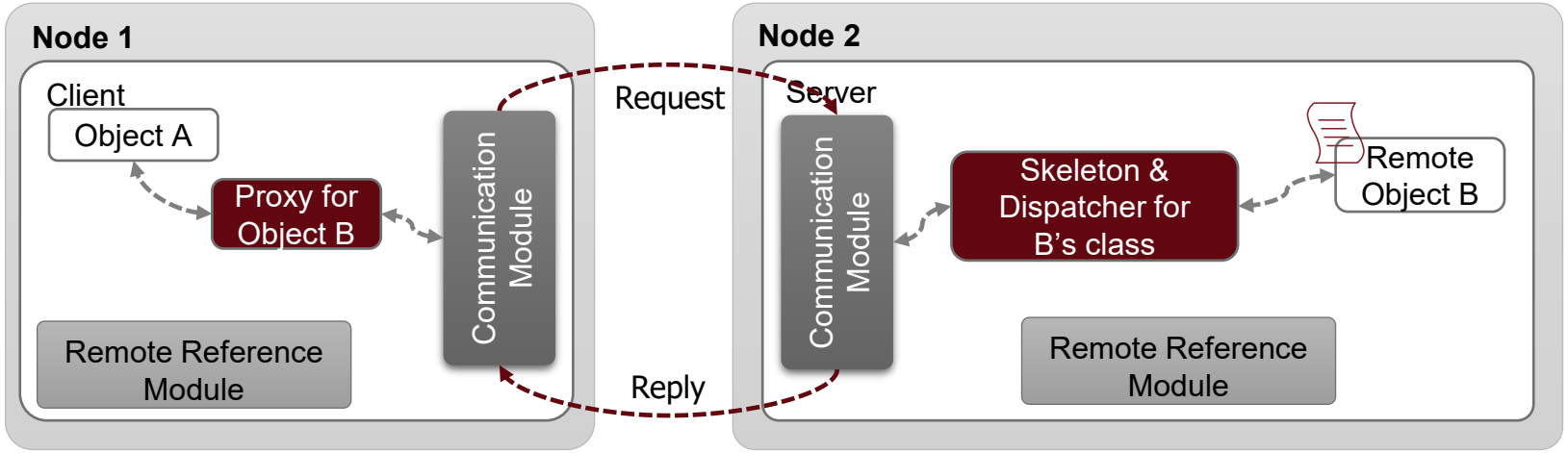
Distributed Object Model – Implementation Aspects

Skeleton: implements methods of the remote interface in a way that the message is decoded and forwarded to the object

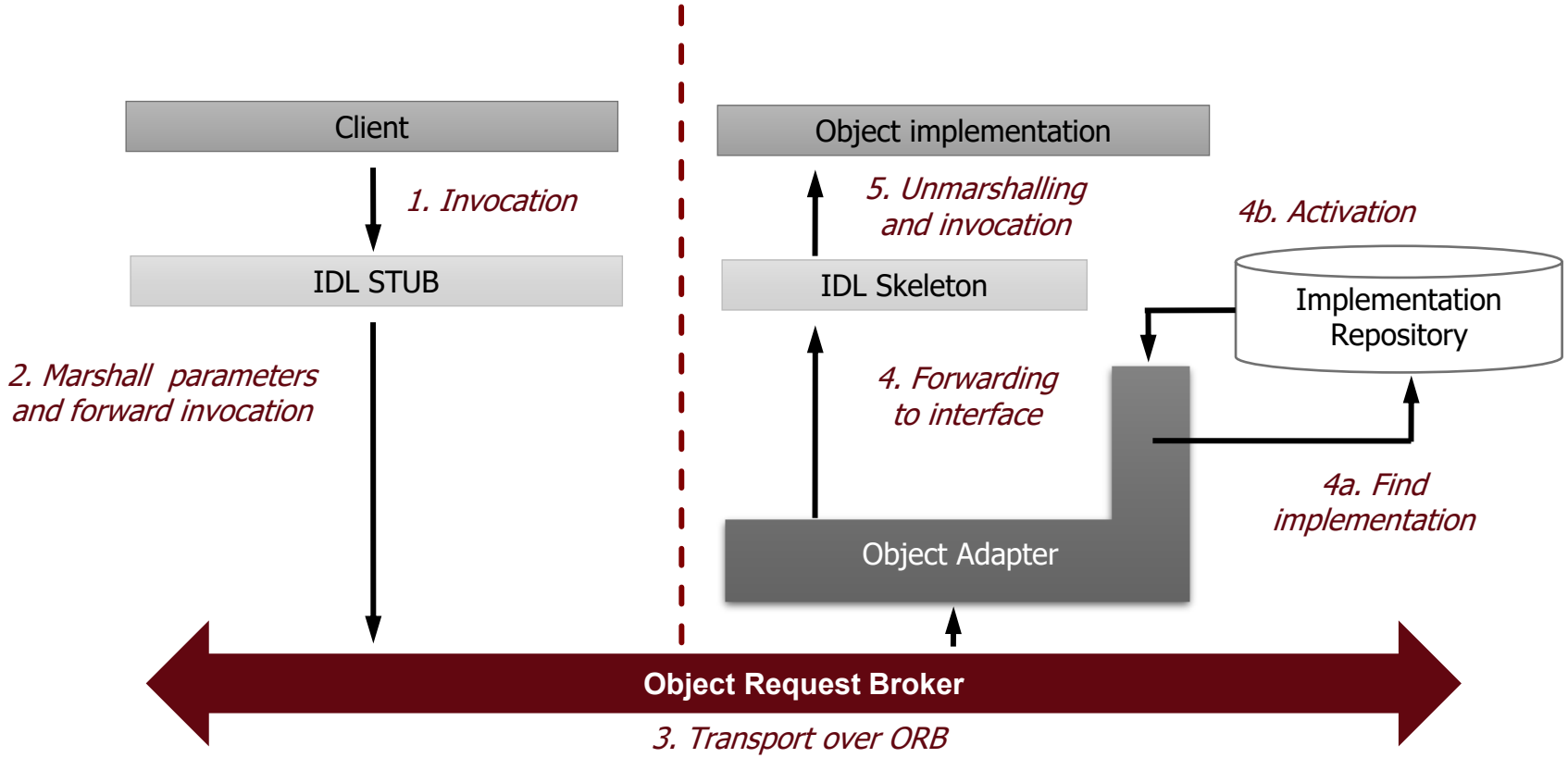


Distributed Object Model – Implementation Aspects

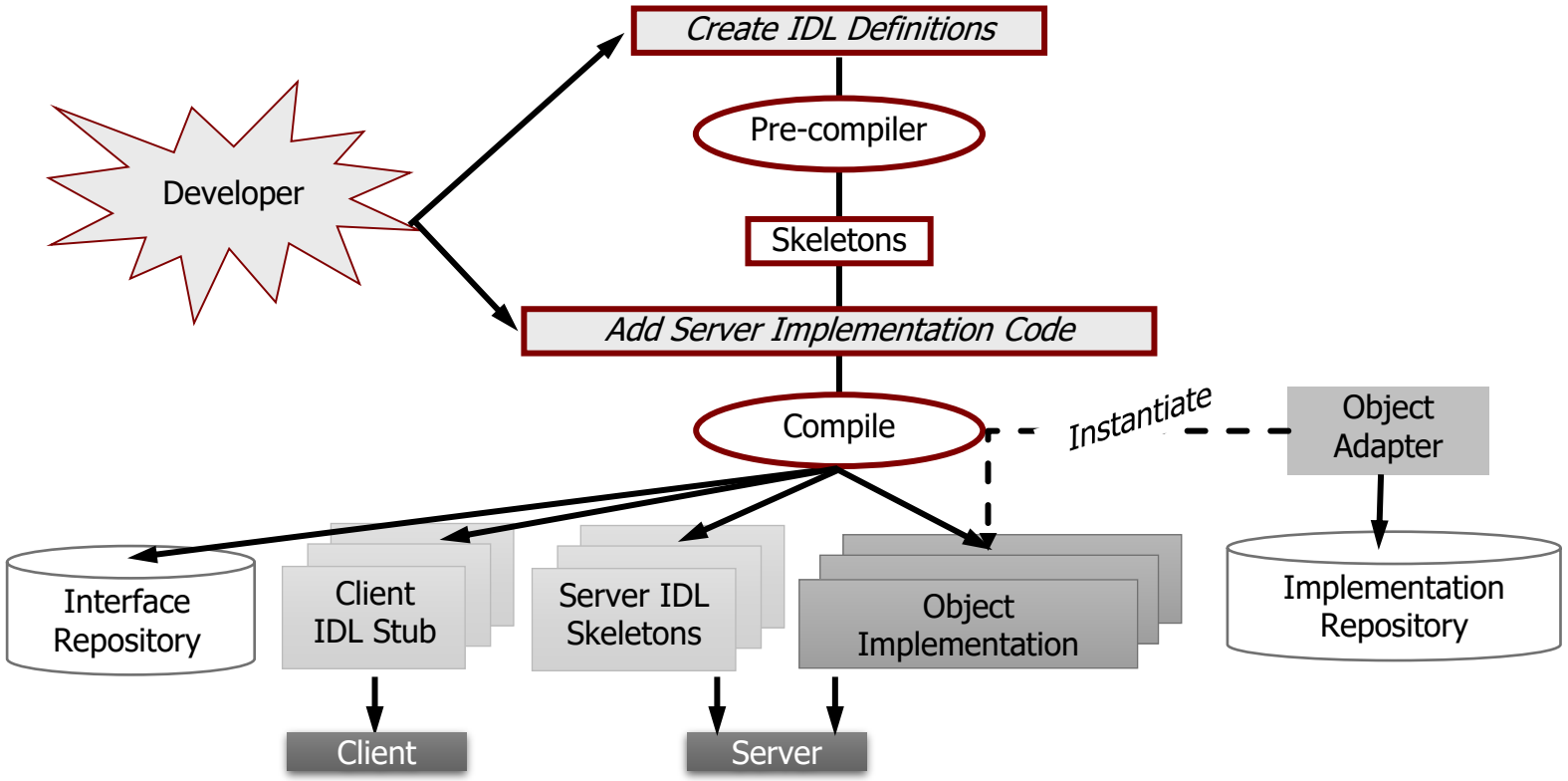
Dispatcher: receives requests from communication module and searches respective method in the skeleton



Static CORBA – Method Invocation



Static CORBA – Method Invocation



Java RMI

Invocation of Java objects generated and managed by another JVM

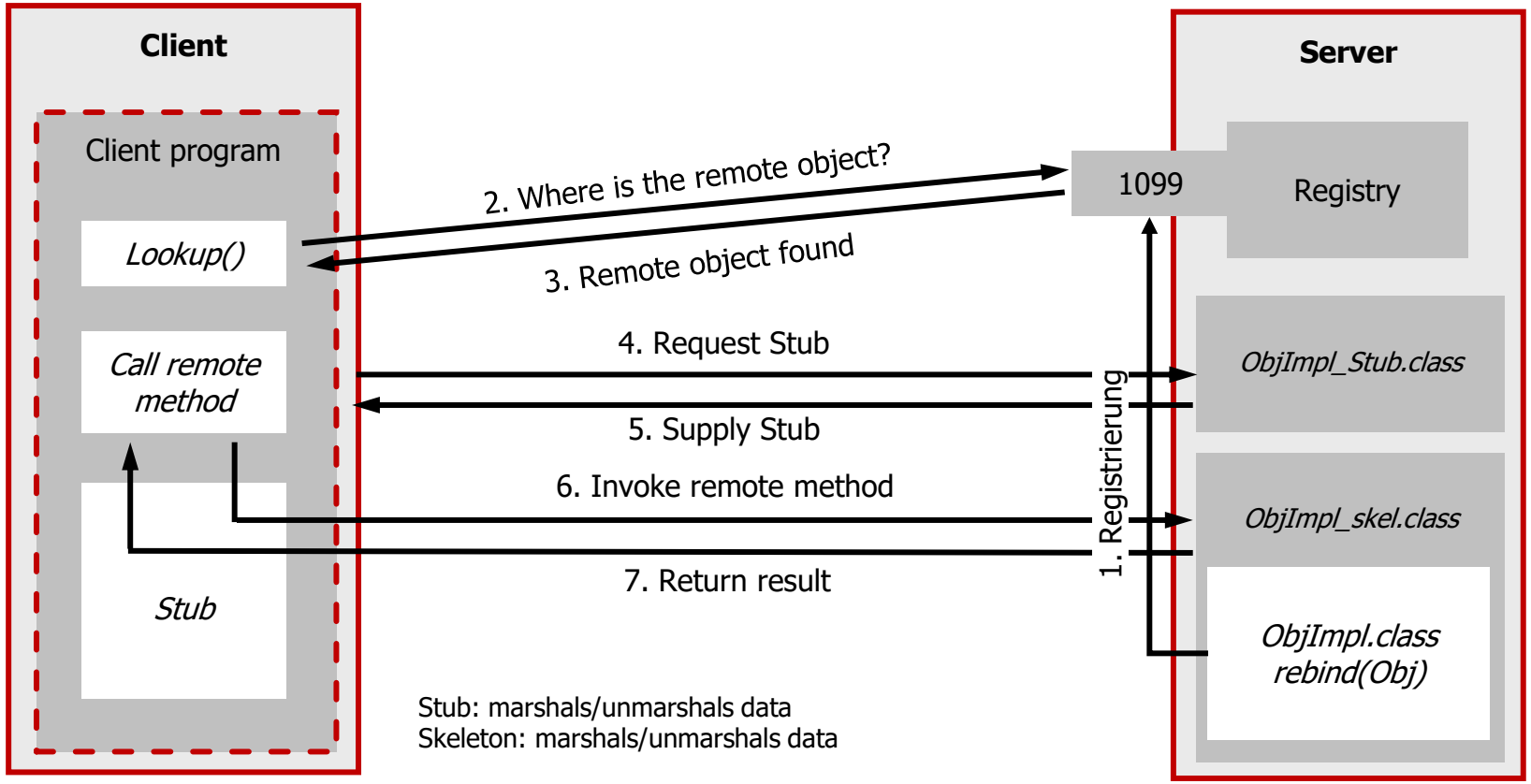
- Java based solution without support for other languages
- Homogeneity (JVM, Java) → no interface definition language required

Communication with remote object from client side over binder (Registry) or handle-driven broker:

- All objects have to be registered in registry, address of registry on Port 1099 as default
- Is the object to be invoked registered in registry with the method?
- If so, stub for remote method is requested
- Method is directly invoked with the stub

https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/hello/hello-world.html

Java RMI – Remote Method Invocation



Java RMI - Programming

Steps required to build a RMI program

- Definition of the remote interfaces (Interface Definition File) describing the remote method to be used. The clients uses this method for interaction with the server.
- Definition of a server application which implements the remote interface (Interface Implementation File)
- Start of registry and server
- Definition and start of client invoking the remote method of the server

Example: Hello World program

- Client requests string from server, Server returns Hello World, Client displays string

Java RMI – Example Hello World

Interface Definition File defines interface which extends standard interface java.rmi.remote

- Subclass of Remote indicates which methods of the remote object can be invoked by a client
- Here only method sayHello()

```
package example.hello;
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote {
        String sayHello() throws RemoteException;
}
```

Java RMI – Example Hello World

Interface Implementation File for class Hello, contains constructor main(), sayHello()

- only sayHello() is available for clients
- Distinction of sayHello() from local methods by declaration in remote interface, method itself remains unchanged
- main() contains server code
- Binding object Hello to Naming Registry by bind (rebind overwrites old entries)

Java RMI – Example Hello World

```
package example.hello;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class Server implements Hello {
    public Server() {}
    public String sayHello() {
        return "Hello, world!";
    }
    public static void main(String args[]) {
        try {
          Server obj = new Server();
          Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 0);
          Registry registry = LocateRegistry.getRegistry(); // Bind the remote object's stub in registry
          registry.bind("Hello", stub);
          System.err.println("Server ready");
        } catch (Exception e) {System.err.println("Server exception: " + e.toString()); e.printStackTrace();
        }
    }}
```

Java RMI – Example Hello World

Generating of stubs and with ~~RMI compiler rmic~~

```
> javac -d destDir Hello.java Server.java Client.java
```

Result

- ~~Skeleton HelloImpl_Skel.class (not necessary since JDK 1.2)~~
- ~~Stub HelloImpl_Stub.class must be available for the client (local or by download) for communication with the server~~

Start of registry (if not started by default in the system)

```
> rmiregistry&
```

Start of server

```
> start java -classpath classDir -Djava.rmi.server.codebase=file:classDir/ example.hello.Server
```

Java RMI – Example Hello World

Client: Access monitoring with security manager, Request to registry with lookup()

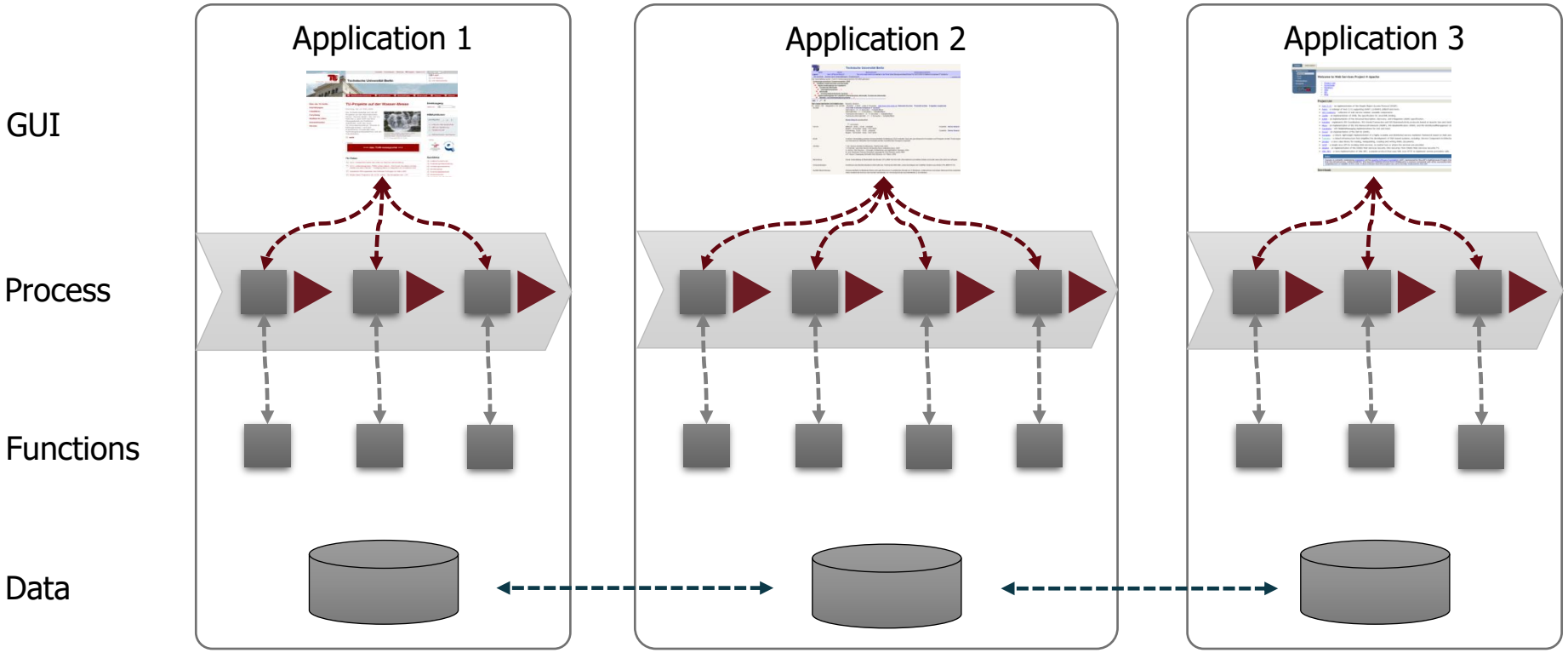
```
package example.hello;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class Client { private Client() {}
public static void main(String[] args)
{
        String host = (args.length < 1) ? null : args[0];
        try { Registry registry = LocateRegistry.getRegistry(host);
        Hello stub = (Hello) registry.lookup("Hello");
        String response = stub.sayHello();
        System.out.println("response: " + response); } catch (Exception e) {
                System.err.println("Client exception: " + e.toString());
                e.printStackTrace();
        } }}
```

java -classpath classDir example.hello.Client

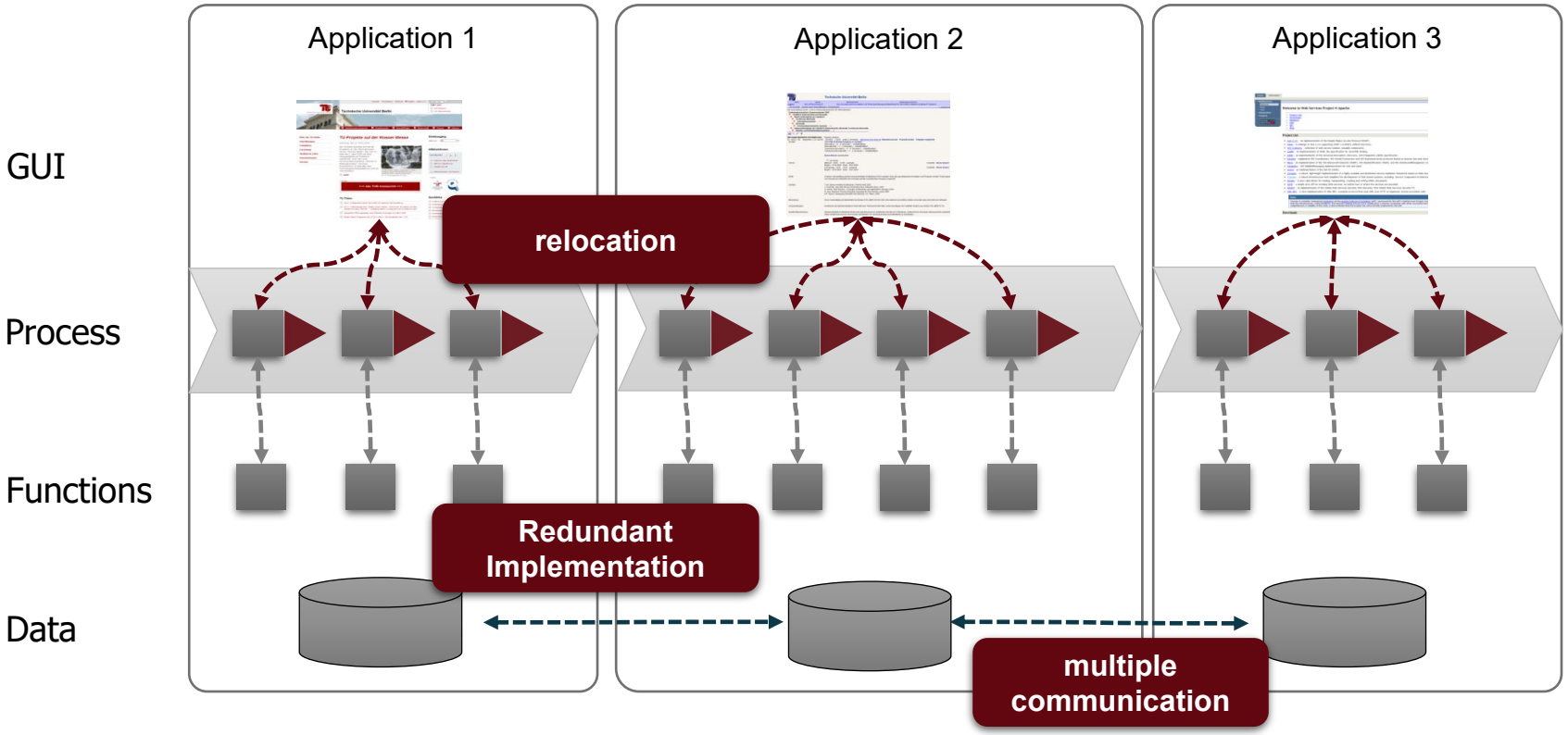
Programming Models

4.2 Service Oriented Architectures (SOA)

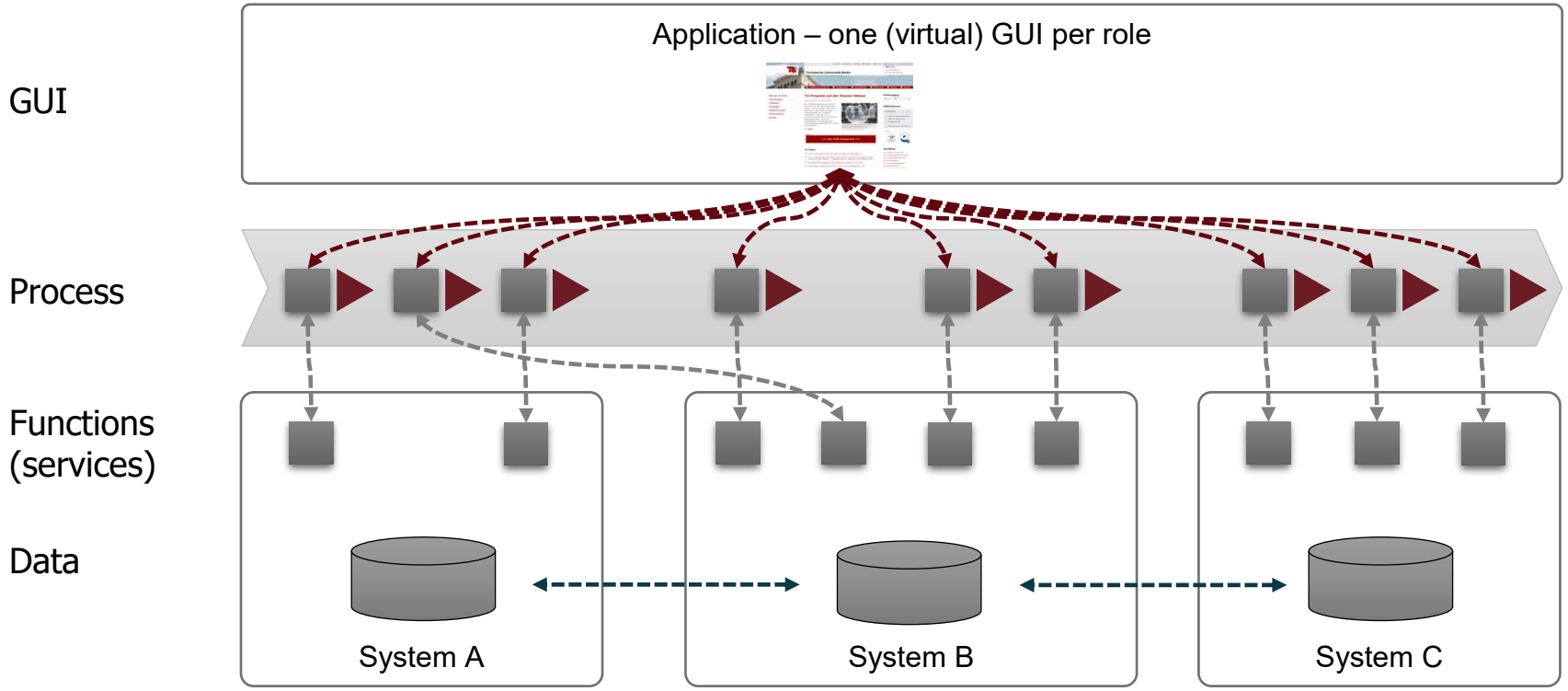
SOA – Motivation: Typical IT Environment



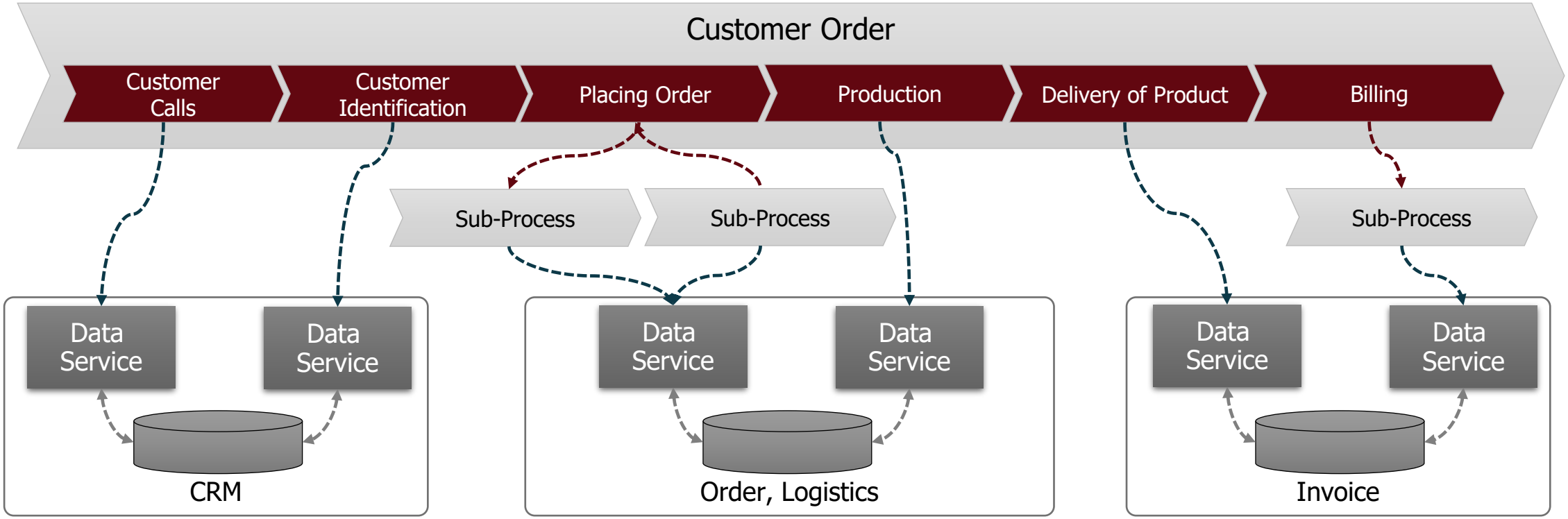
SOA – Motivation: Typical IT Environment



SOA – Motivation: Typical IT Environment



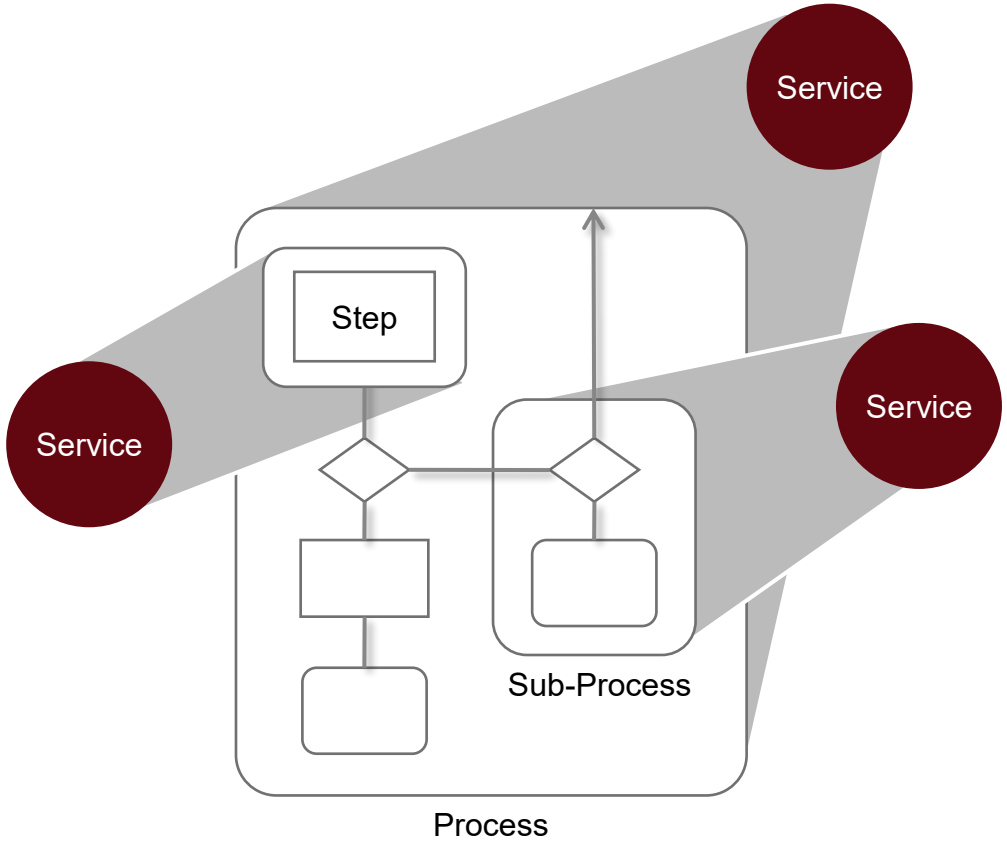
SOA – Motivation: Business Process - Example



SOA - Properties

Services (in SOA) …

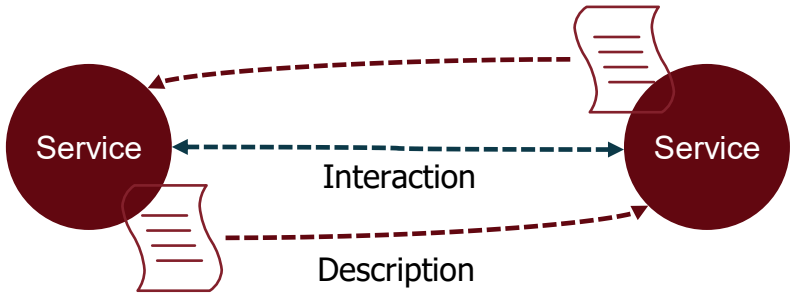
– Encapsulate functionality



SOA - Properties

Services (in SOA) …

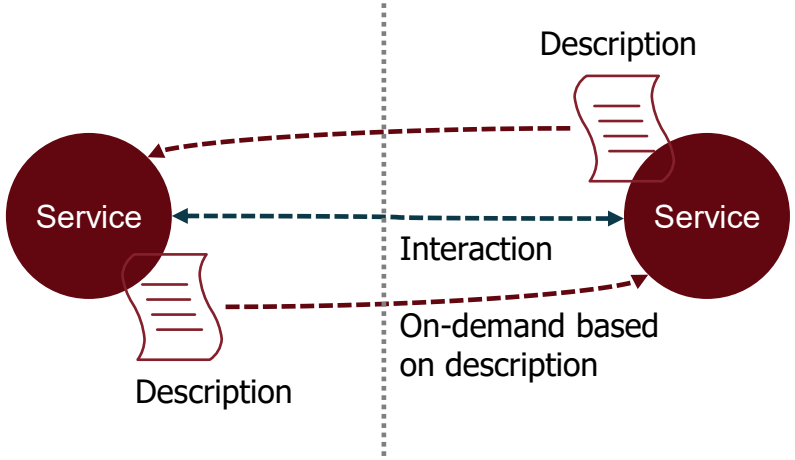
- Encapsulate functionality
- Have well-defined interfaces



SOA - Properties

Services (in SOA) …

- Encapsulate functionality
- Have well-defined interfaces
- Are loosely coupled



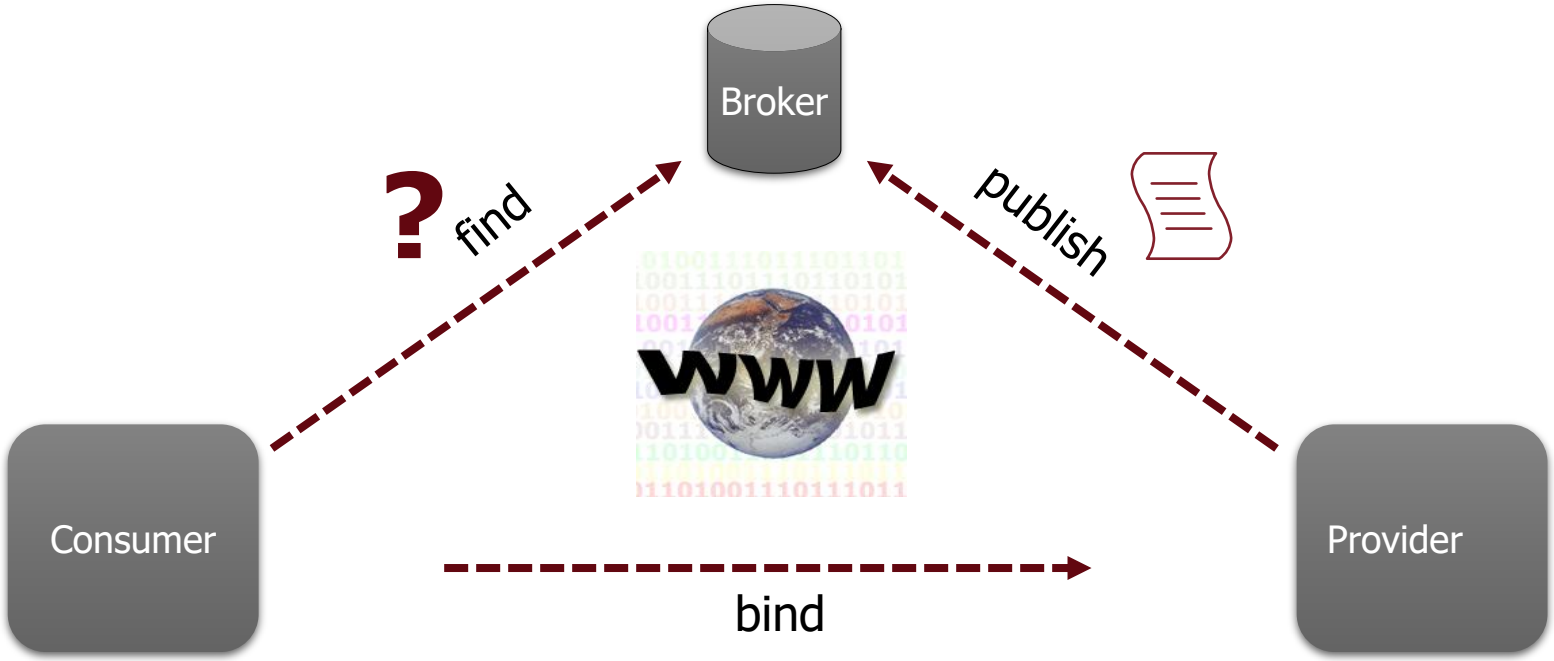
SOA - Properties

Services (in SOA) …

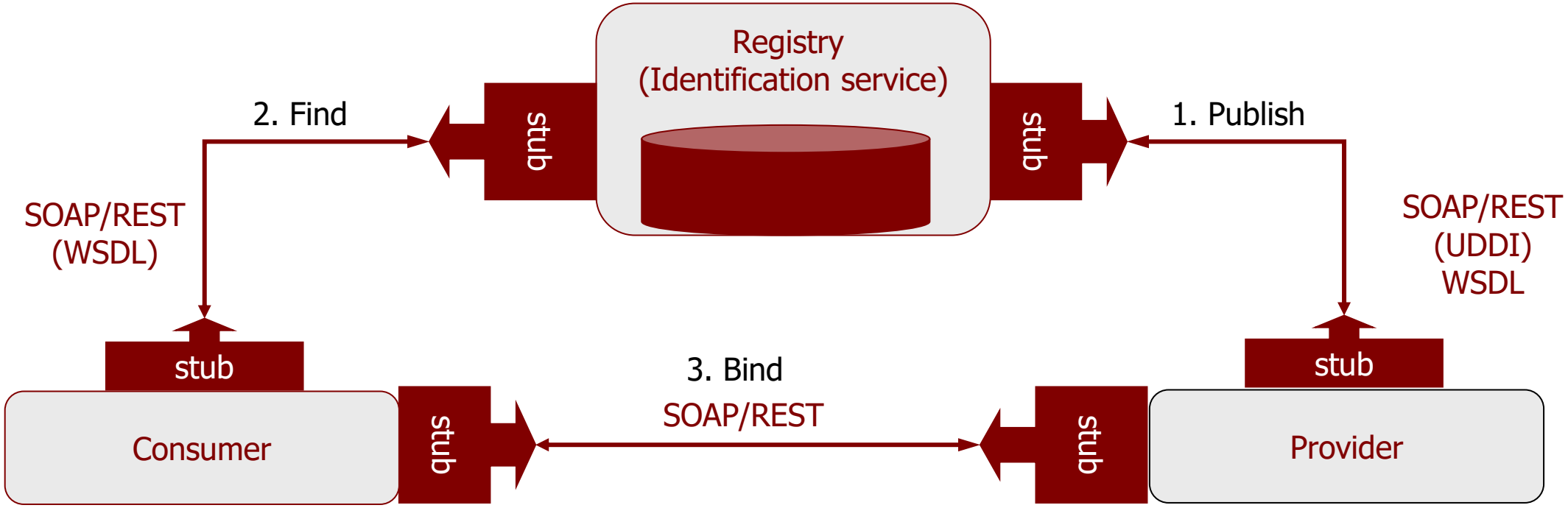
- Encapsulate functionality
- Have well-defined interfaces
- Are loosely coupled
- Are independent and autonomous pieces of software
- Interact using messages
- Are used to implement business processes
- Provide means for workflow control

Pattern: SOA Triangle

SOA is an architecture pattern following the publish, find, bind model …



Web Services: SOA Triangle SOAP ↔ (WSDL ↔ UDDI)



APIs (Application Program Interfaces)

APIs allow applications to communicate with one another: on the same computer, over a local network, or over the internet

API = contract between a client application and a service application

- Client sends a request in specific format to API of service
- Service API sends a response back to the client in an agreed upon format
- Neither client nor service need to know the implementation details of the other

APIs allow access to resources while maintaining security and control

Different types of Web APIs

Remote Procedure Call, RPC

- Clients can call functions on the server

Remote Method Invocation, RMI

- Clients can call methods on objects on the server

Representational State Transfer, REST

- Clients can apply CRUD (Create, Read, Update, Delete) operations on resources on the server

Example tools for building, testing, documenting APIs

Postman

Testfully
Insomnia
Hoppscotch
Apidog
Postcode
Rapid API
Firecamp
TestMace
Nightingale
API Tester
curlx
RecipeU

SOAP (Simple Object Access Protocol)

Description of remote method calls with XML

SOAP as a simple way to deliver structured information over HTTP to application

- No specification of call semantics, just marshalling and packaging of the data
- One-way communication
- Standardization at W3C

SOAP messages = text files sent over existing network infrastructure such as HTTP or SMTP

SOAP defines

- How to translate a method call with endpoint, method name and parameters into XML
- How to convert the result to XML
- In which format the data is to be transmitted over the network

Structure of SOAP

SOAP messages defined by XML schema components

Envelope: Frame with a description of the message content and information about processing. Information structured in

- Header: optional area with information about the process, authentication (username, password), transaction information, routing...
- Body: the data to be transported
- SOAP encoding with optional information for displaying the data/objects

Sender and receiver must agree on how to interpret the data in the SOAP header

- Document: Format clarified in advance between partners ⇒ any XML documents
- rpc: Structure of body section defined in SOAP standard (method name, parameters, ...)

Example SOAP (MathCalc int a + int b)

```
POST /WebService_technik/Service1.asmx HTTP/1.1
Host: localhost
Content-Type: text/xml; charset=utf-8
Content-Length: length
SOAPAction: "http://tempuri.org/MathCalc"
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" …
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <MathCalc xmlns="http://tempuri.org/">
      <a>int</a>
      <b>int</b>
    </MathCalc>
  </soap:Body>
</soap:Envelope>
```

Values inserted during creation of the message

Example SOAP (MathCalc int a + int b)

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: length

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <MathCalcResponse xmlns="http://tempuri.org/">
      <MathCalcResult>int</MathCalcResult>
    </MathCalcResponse>
  </soap:Body>
</soap:Envelope>
```

Request and reply with HTTP Post

```
POST /WebService_technik/Service1.asmx/MathCalc HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded
Content-Length: length

a=int&b=int
```

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: length

<?xml version="1.0" encoding="utf-8"?>
<int xmlns="http://tempuri.org/">int</int>
```

REST and SOAP

Two approaches to building APIs, data transmission between web applications

Main difference: SOAP is an official protocol maintained by W3C, REST is a set of architectural principles (not a protocol!)

- Many legacy systems still adhere to SOAP
- SOAP web services offer built-in security and transaction compliance that align with many enterprise needs, but that also makes them heavier.
- REST APIs came later and often viewed as a faster alternative → used in web-based scenarios, IoT, mobile applications, and serverless computing.

URI vs. URL IS ONE OF THE MOST FAMOUS GEEK BATTLES, BUT IT'S SIMPLER THAN PEOPLE MAKE IT OUT TO BE.

- BOTH URLS AND URIS ARE **ways of finding something** ONLINE
- A URI CAN BE A NAME BY ITSELF, **OR** A NAME WITH A PROTOCOL
- A URL IS A *subtype* OF URI WITH BOTH A RESOURCE **and a protocol**
- IN OTHER WORDS, **all URLs are URIs, but not all URIs are URLs**

What is REST?

Architectural style for distributed hypermedia systems (Roy Thomas Fielding, 2000)

- REST server provides access to resources via HTTP methods
- REST APIs define functions for developers to perform requests and receive responses
- Stateless: client's responsibility to maintain state and pass this state with each request
- Resource is identified by a Uniform Resource Identifier (URI) (see e.g. https://danielmiessler.com/p/difference-between-uri-url/)
- Code on demand: server sends back static representations of resources described with XML or JSON. However, when necessary, servers can send executable code to the client.

Department of Software Engineering, Rochester Institute of Technology
Peter Larsson-Green, Jönköping University

What does REST mean?

The name "Representational State Transfer" is intended to evoke an image of how a well-designed Web application behaves: a network of web pages (a virtual state-machine), where the user progresses through the application by selecting links (state transitions), resulting in the next page (representing the next state of the application) being transferred to the user and rendered for their use.

REST is an architectural style, not a specification → used in many different ways

REST vs. RESTful: RESTful API = API built according to REST architecture.

Why REST?

Maintains separation between client and server: same interface used whether the client is a user interface or another REST API server

Provides a uniform interface to access and manage resources

Scalability/Reliability: REST APIs can be deployed to multiple servers in different locations

- Fail-over: If server unavailable, requests automatically routed to another server
- Load balancing: With increasing request volumes, additional REST API servers deployed

Language and Platform Independence

- REST APIs can be written in many languages, clients can be written in a completely different language
- REST APIs can be hosted on nearly any Operating System
- REST APIs accept and return multiple data formats, e.g. JSON, XML

What does REST mean?

Client

Server

GET /users/2
...

{"id": 2, "name": "Bob"}

Changes state.

{"id": 2, "name": "Obi"}

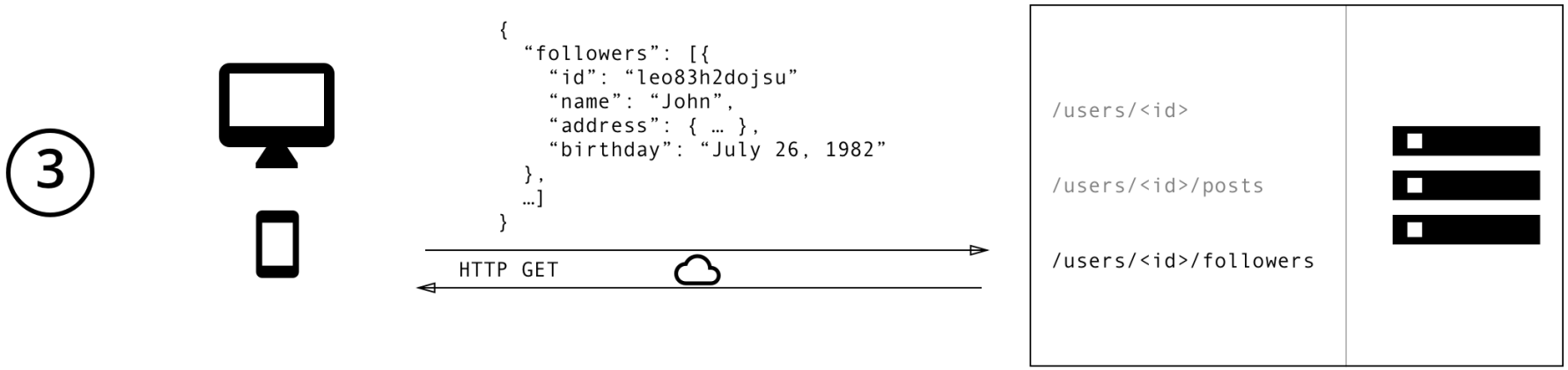
PUT /users/2
{"id": 2, "name": "Obi"}

Id	Name
1	Alice
2	Bob
3	Claire

Users

What does REST mean?

What does REST mean?



REST Request Components

Method: Required, identifies the operation (Example: GET)

URI: Required, Identifies the resource, may include query parameters to identify specific content or actions (Example: http://localhost:8080/jedi/5)

Headers: Optional, but generally used, additional information about the request or client, e.g. browsers often add standard and their own headers to requests

Body: used for POST and PUT, but not for GET and DELETE, Representation of one or more objects (Example: {"id": 3, "lastName": "Skywalker", "firstName": "Anakin"})

REST Response Components

Status Code: Required, identifies the result of the operation

- Example: 200/OK

Headers: Optional, provides additional information about the response to the client

- Example: api-status-code: 3

Body: required for GET, but often used for other methods, representation of one or more objects

- Example: {"id": 3, "lastName": "Skywalker", "firstName": "Anakin"}

URIs

A resource is identified by a Uniform Resource Identifier

scheme	://	host:port	/	path to resource	?	query parameter
http	://	www.tu.berlin:9150	/	eecs/faculty	?	id=310

Using HTTP as the uniform interface

HTTP methods specify operation

- Create: POST (or PUT)
- Retrieve: GET
- Update: PUT (or PATCH)
- Delete: DELETE

```
POST /login-sessions
POST /books
GET /top-10-books
```

HTTP headers: Content-Type and Accept specify data format for the resources.

HTTP status code indicated success/failure.

Examples online

- https://restful-api.dev/
- https://reqbin.com/rest-api-testing

REST example

HTTP Method	DELETE
URI	http://localhost:8080/UserManagement/rest/UserService/users/1
Operation	Delete User with Id 1

HTTP Method	POST
URI	http://localhost:8080/UserManagement/rest/UserService/users/2
Operation	Insert user with Id 2

HTTP Method	GET
URI	http://localhost:8080/UserManagement/rest/UserService/users
Operation	Get list of users

HTTP Method	GET
URI	http://localhost:8080/UserManagement/rest/UserService/users/1
Operation	Get user of Id 1

HTTP Method	PUT
URI	http://localhost:8080/UserManagement/rest/UserService/users/2
Operation	Update User with Id 2

REST example

GET method to retrieve resources.

Which data format? Specified by the Accept header!

```
GET /users HTTP/1.1
Host: the-website.com
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 66
[
  {"id": 1, "name": "Alice"},
  {"id": 2, "name": "Bob"}
]
```

REST example

POST method is used to create resources.

Which data format? Specified by the Accept and Content-Type header!

```
POST /users HTTP/1.1
Host: the-website.com
Accept: application/json
Content-Type: application/xml
Content-Length: 49
<user>
  <name>Claire</name>
</user>
```

```
HTTP/1.1 201 Created
Location: /users/3
Content-Type: application/json
Content-Length: 28

{"id": 3, "name": "Claire"}
```

REST example

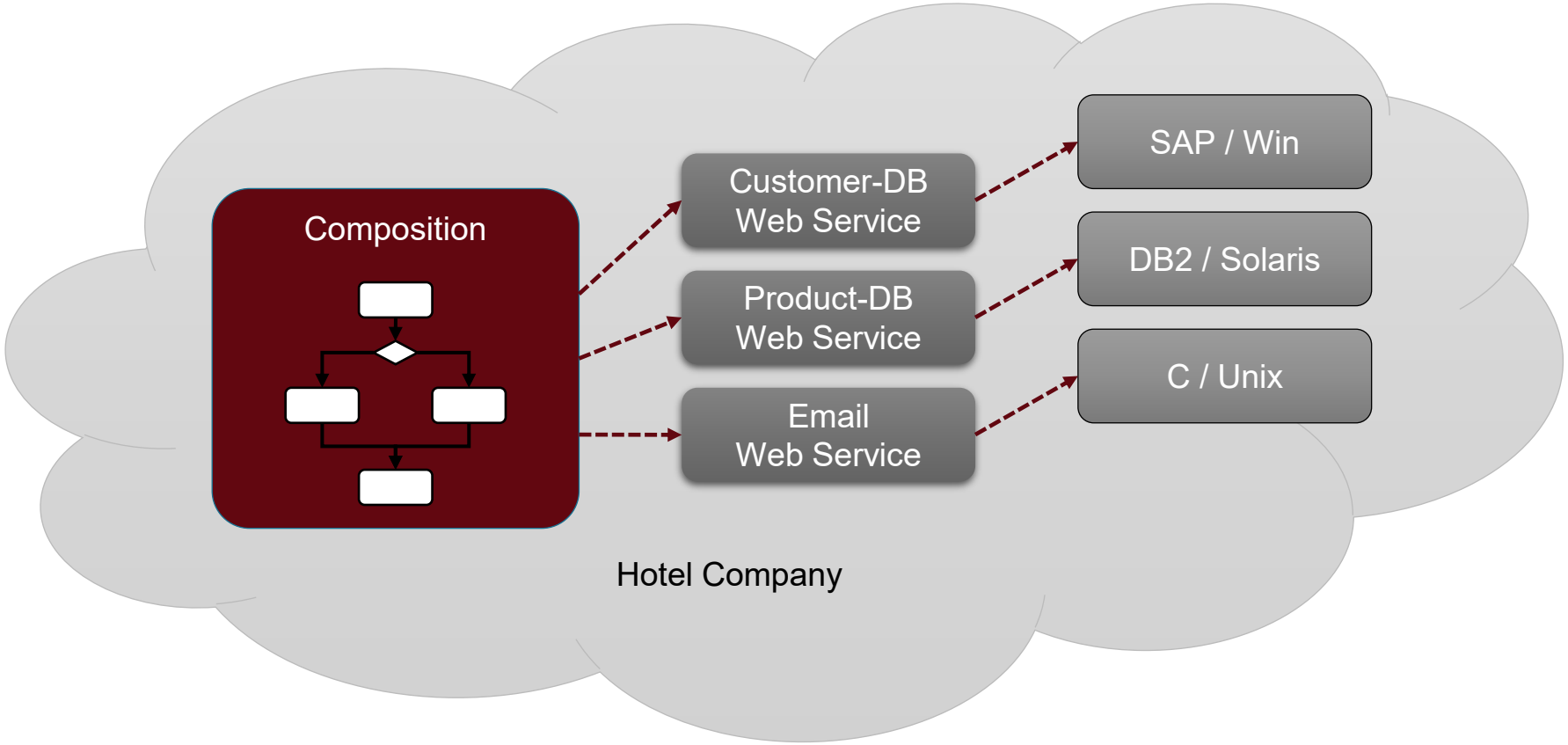
HTTP status codes to indicate success/failure.

- 200/OK – Request was completed successfully
- 201/Created – Resource was created successfully
- 400/Bad Request – Body of request was invalid
- 403/Forbidden – Caller does not have permissions for the requested resource
- 404/Not Found – Requested resource could not be found
- 500/Internal Server Error – Server cannot fulfill request

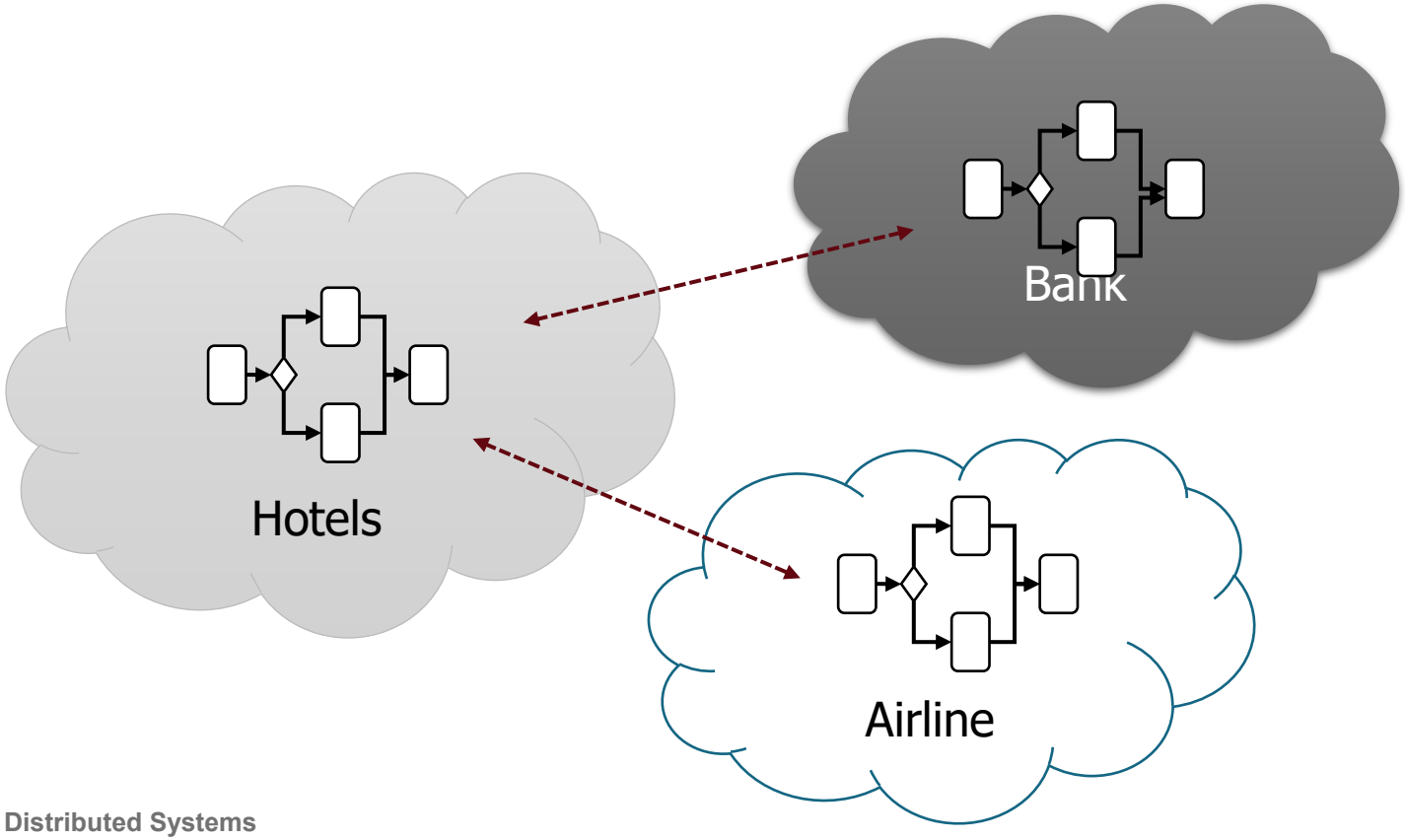
```
GET /users/999 HTTP/1.1
Host: the-website.com
Accept: application/json
```

```
HTTP/1.1 404 Not Found
```

SOA – Workflows: EAI (Enterprise Application Integration)



SOA – Workflows: B2B-Integration



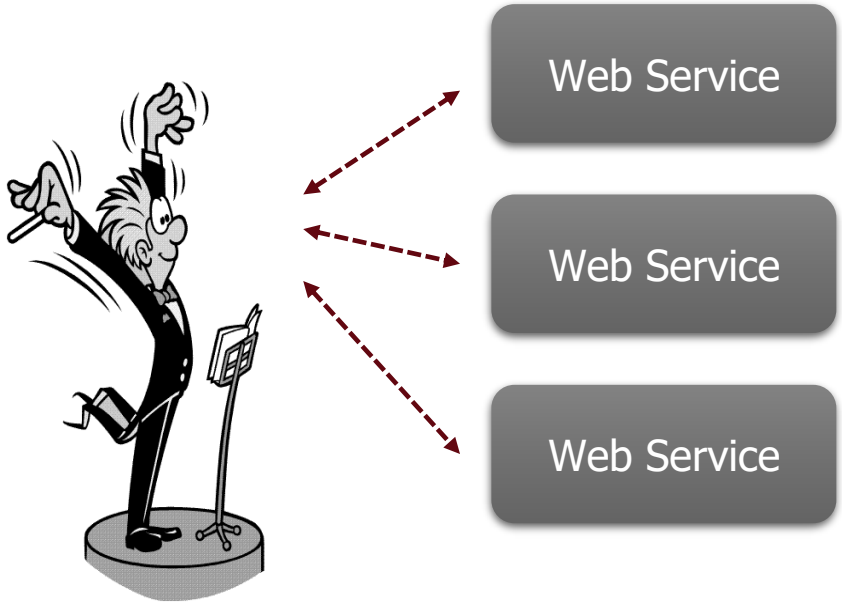
SOA – Workflows

Web Service Orchestration (WSO)

- Composition of web services for business processes
- Definition of processes that reuse existing web services
- Aim: Declarative (non-programmatic) description of composed service

Defined elements

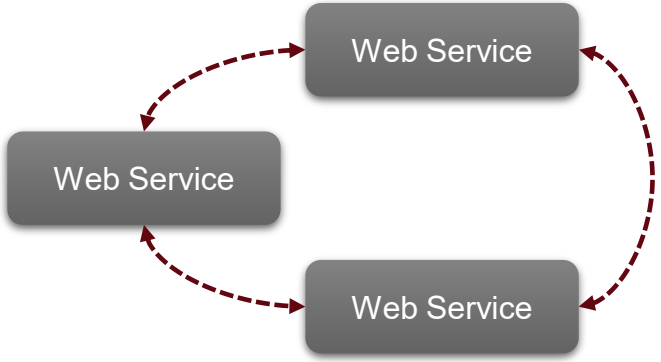
- Services to be executed
- Order of execution
- Parallelism and
- Conditional execution



SOA – Workflows

Web Service Choreography (WSC)

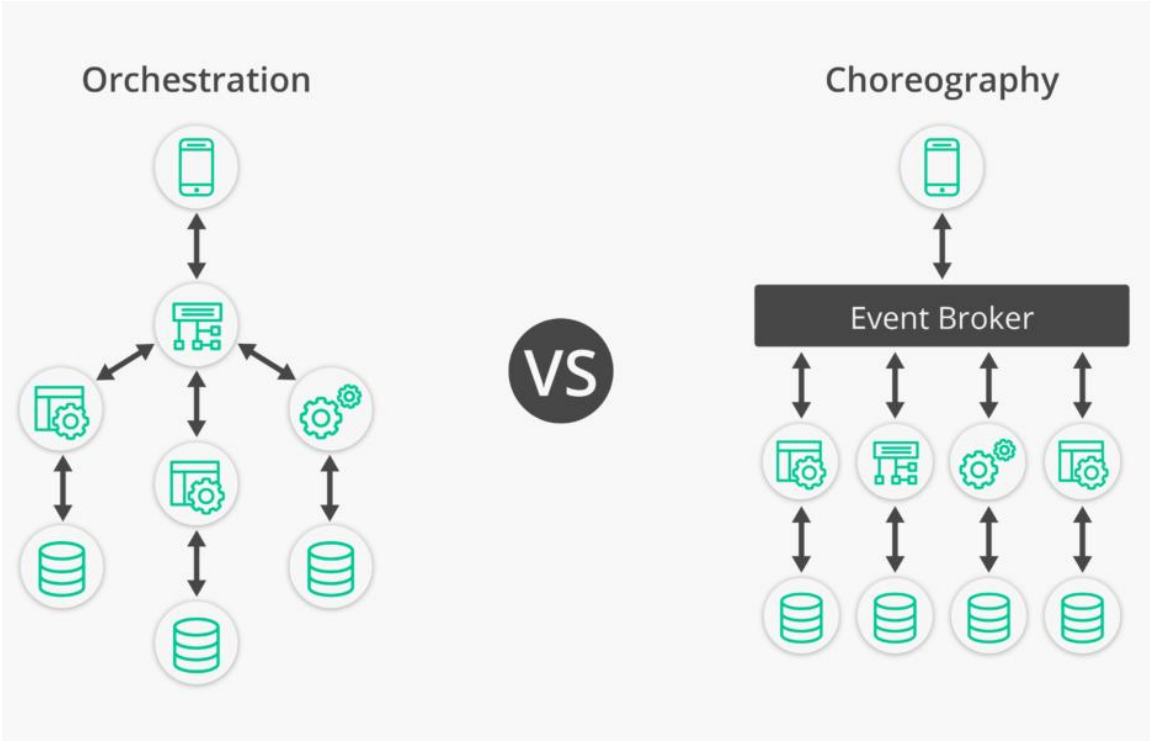
- Composition of web services for business collaboration
- Definition of peer-to-peer collaboration of several partners
- Difference to business process carried out by only one partner (at WSO): Partners can describe their part of the interaction (exchange of messages between partners)
- Necessary: Formal description of message exchange that models the peer-to-peer exchange



Orchestration vs. Choreography

Orchestration entails actively controlling all elements and interactions like a conductor directs the musicians of an orchestra

Choreography entails establishing a pattern or routine that microservices follow as the music plays, without requiring supervision and instructions.



Microservices

"*In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.*"

James Lewis and Martin Fowler

Microservices

Fundamental Properties:

- Componentization via services
- Smart endpoints and dumb pipes
- Decentralized data management, Decentralized governance
- Infrastructure automation (CI/CD)
- Design for failure
- Products, not projects - "you build, you run it"

vs. SOA?? → Different points of view: “Microservices is SOA done well“

SOA uses services to integrate multiple applications

Microservices use services to structure a single application

Microservices vs. SOA

	SOA	Microservices
Implementation	Different services with shared resources.	Independent, purpose-specific services.
Communication	ESB uses multiple notification protocols such as SOAP, AMQP and MSMQ.	APIs, Java Message Service, Pub/ Sub
Data storage	Shared data storage.	Independent data storage.
Deployment	Challenging. A complete rebuild is required for small changes.	Easy deployment. Any microservice can be containerized.
Reusability	Reusable services through shared resources.	Each service has own independent resources. Reusing through APIs.
Speed	Slows down as more services are added.	Constant speed as traffic increases.
Governance flexibility	Consistent data governance across services.	Different data policies for each storage.

Programming Models

4.3 Event-driven Architecture (EDA)

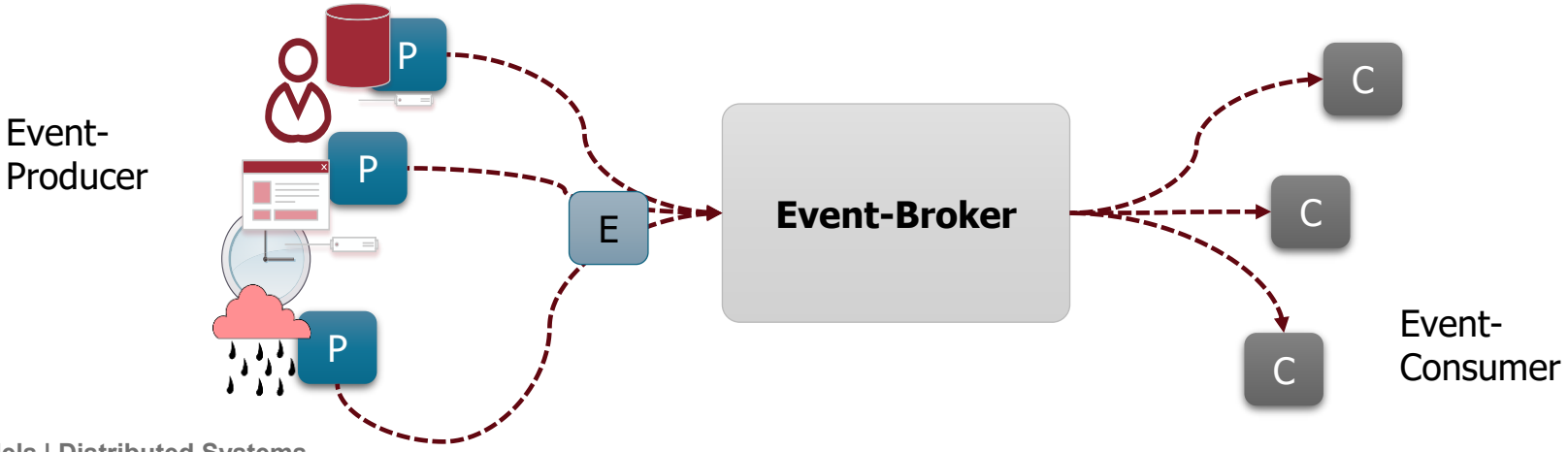
Motivation for Publish/Subscribe Systems

Mechanisms to access static data (queries, services) are different from those controlling the flow of information (subscriptions/filters, event handlers)

Data generated by many internal and external data sources

Extreme loose coupling, data sources have no knowledge about data sinks

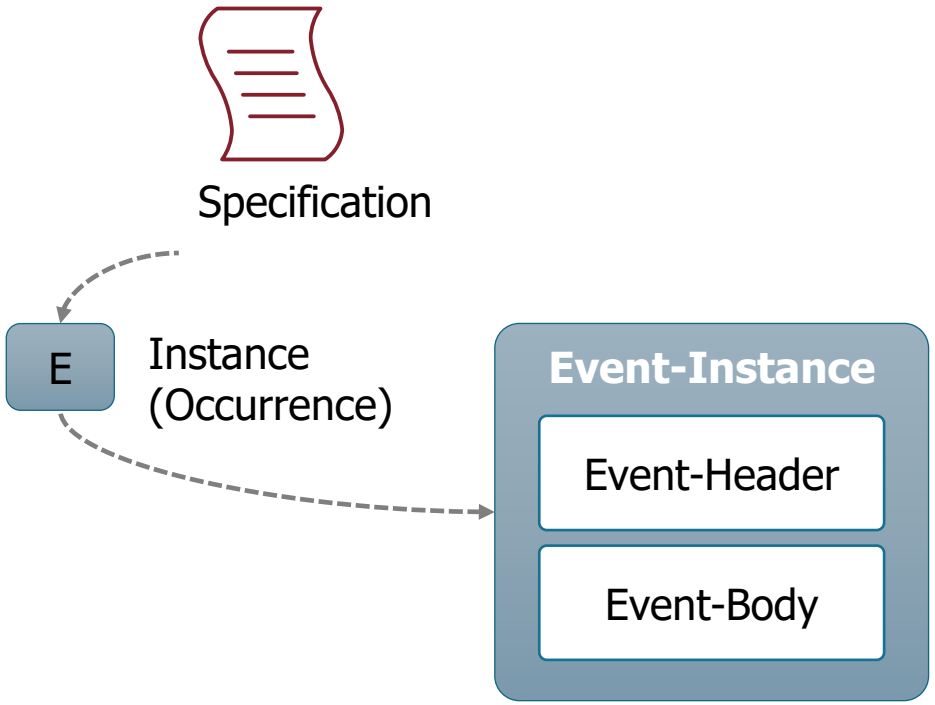
Interaction happens spontaneous, triggered externally → analogy to human nerve system



Definition of Event

Event: Single state-transforming action conducted by a process, whereas the state refers to the values of all variables of the process

Informal: something happened inside or outside our application (e.g. problem occurred, threshold reached, deviation from expected behavior observed)



Components: Event Generators

Every event has a source, e.g. application, data store, service, user, sensor

- Generator (producer) is either part of the source or observes the source
- Processing an event may lead to the generation of a new event

Event-Generator
(Event-Producer)

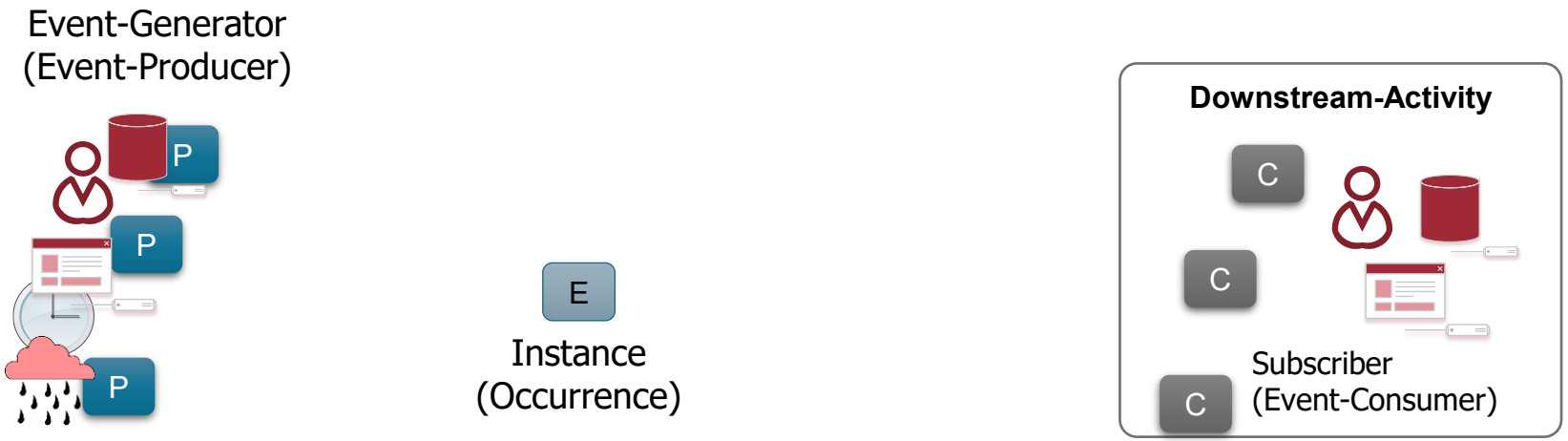
E

Instance
(Occurrence)

Components: Downstream Activity (Subscriber)

A single event can initiate several *Downstream* actions/activities:

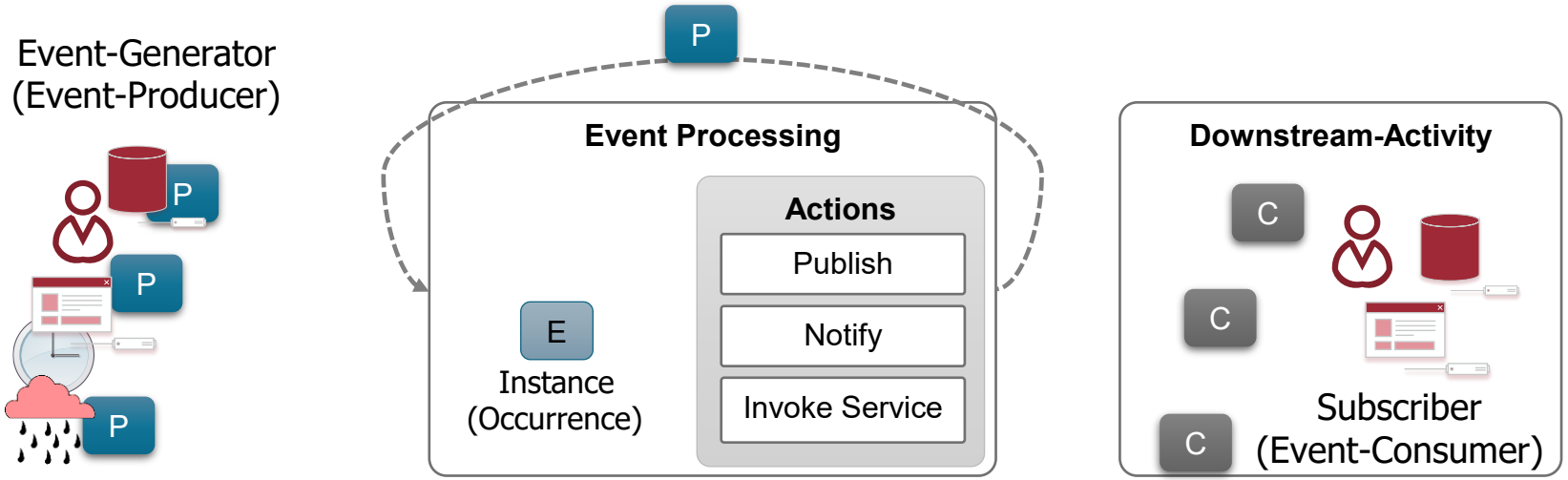
- A pull by subscribers of event publications (e.g. humans, applications, services, ..)
- A push resulting in a service invocation, notification, business process initiation
- But who pushes, from whom is pulled?



Components: Event Processing

Event Processing Layer evaluates events and initiates actions

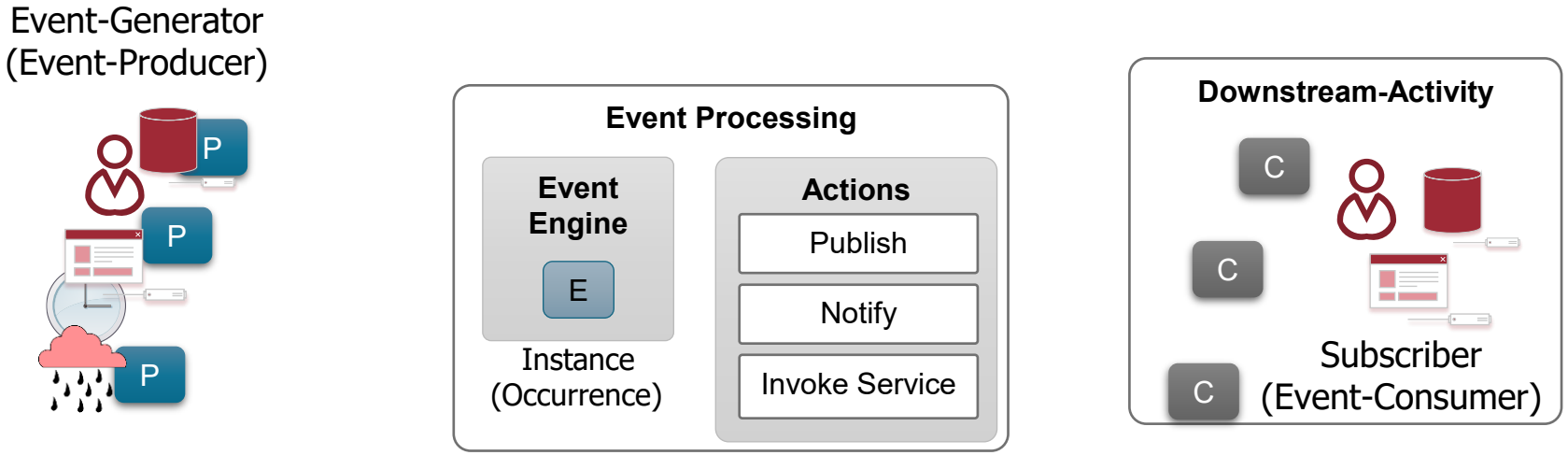
- Subscribers (interested parties) define event processing rules used to evaluate events
- Rules and evaluation results are linked to actions
- Invoke a service, publish event to subscribers, generate new events, …



Components: Event Processing

Event Processing Layer evaluates events, initiates actions processed by engines:

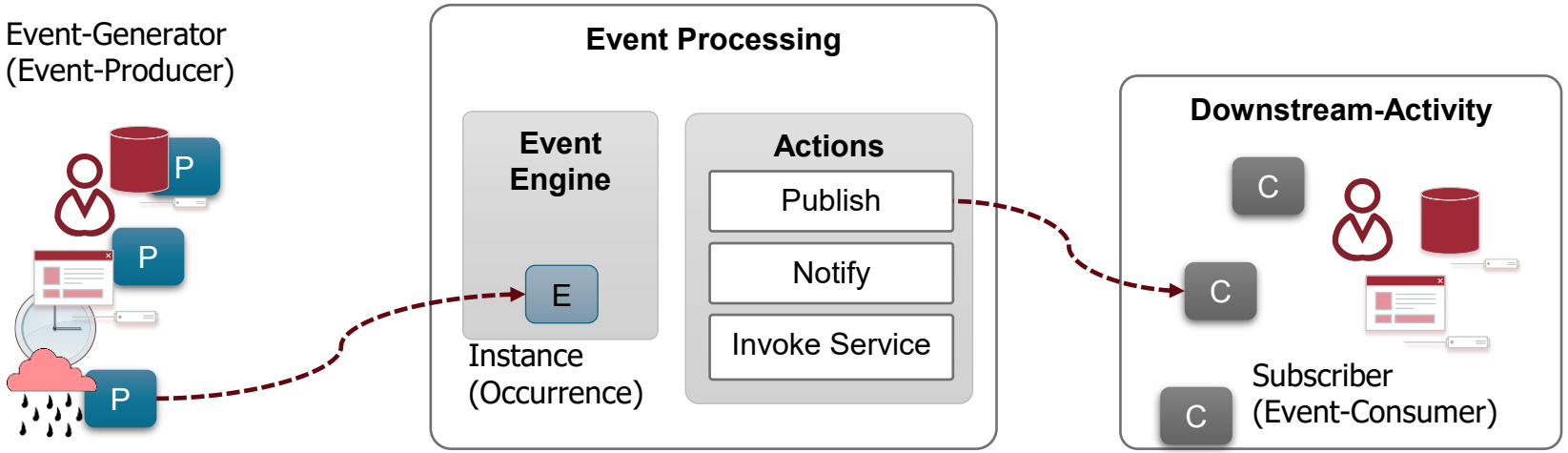
- Simple engines process each event instance independently
- Complex engines process events in context of prior events



Components: Event Channel

Messaging backbone required to transmit events from generators to downstream-activities via the event processing layer

Typical event channel provider: message queues, publish/subscribe systems



Event Processing Styles

Simple Event Processing: Notable events happen and initiate actions

- Primary purpose: optimize (real-time) information flow in the systems

Stream Event Processing: Both notable and ordinary events happen

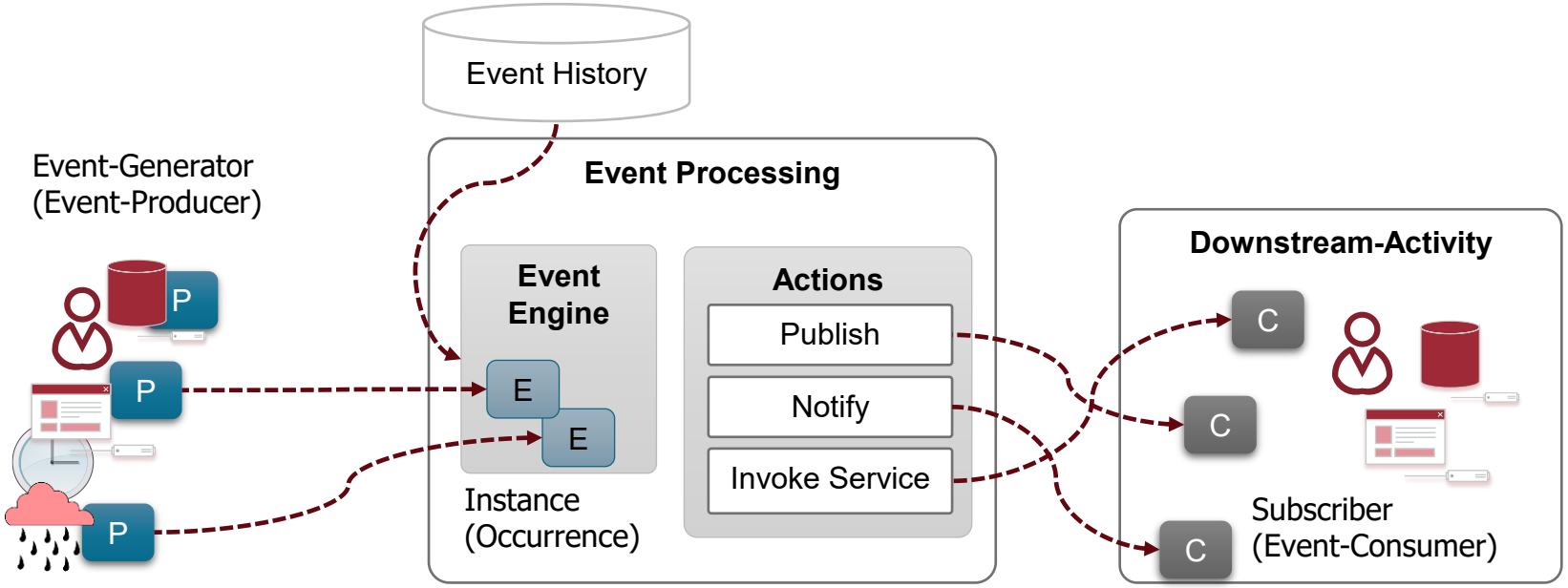
- Ordinary events (regular state transitions of an application – e.g. a customer order) are screened for notability and streamed to subscribers
- Primary purpose: enables for decision making

Complex Event Processing: Processing an event includes considering context information gathered through previous events

- Events can be aggregated and correlated (e.g. causal, spatial, temporal)
- Primary purpose: detect and respond to anomalies, threats, opportunities

Complex Event Processing: Actions

- Event-pattern detection
- Event abstraction
- Event filtering
- Event aggregation and transformation
- Modeling event hierarchies
- Detecting relationships (causality, membership, timing) between events



Programming Models

4.4 Agentic AI

Agentic AI

Intelligent agents in core of AI applications = autonomous entities that choose responses and perform tasks based on their environment

Agents range from simple rule-based systems to advanced learning systems powered by LLMs, classified based on

- Level of intelligence
- Decision-making processes
- Interaction with surroundings to reach outcomes

Five main types of AI agents: simple reflex agents, model-based reflex agents, goal-based agents, utility-based agents and learning agents

(see https://www.ibm.com/think/topics/ai-agent-types for details)

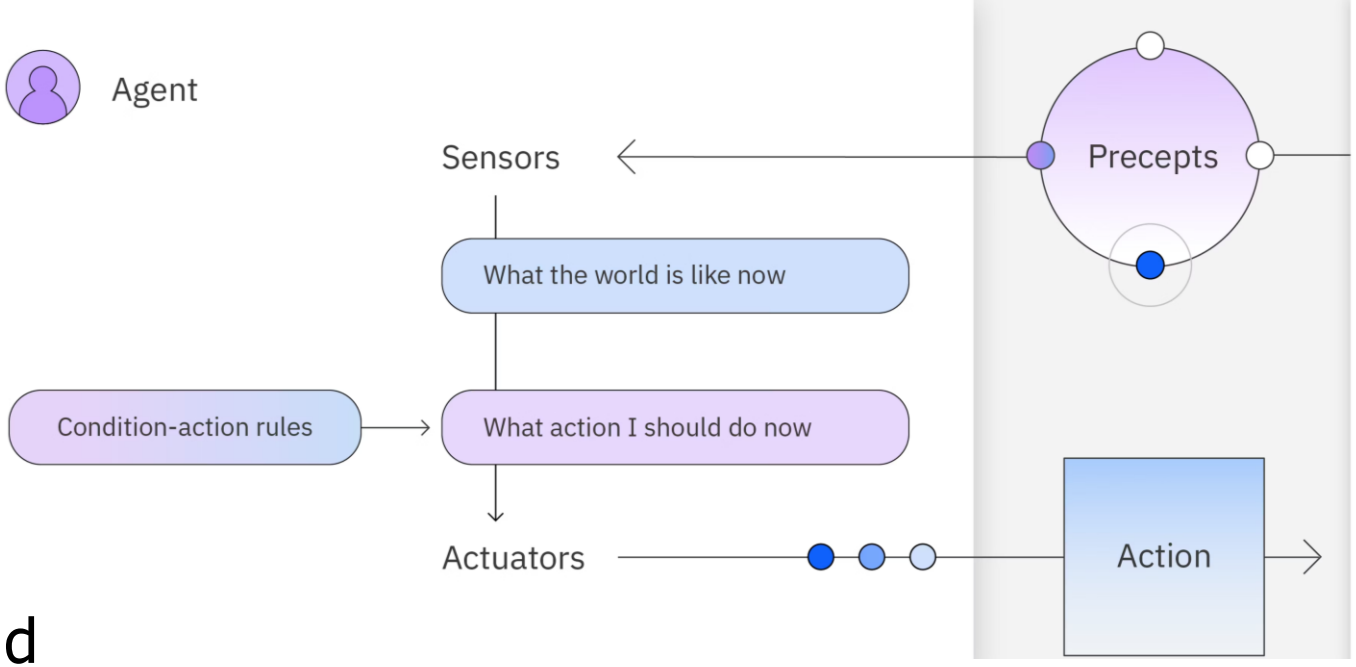
Simple reflex agents

Operate based on direct responses to environmental conditions

Follows predefined (condition-action) rules, determines actions without considering past experiences or future consequences

Example: thermostat turns on heater if temperature drops below threshold

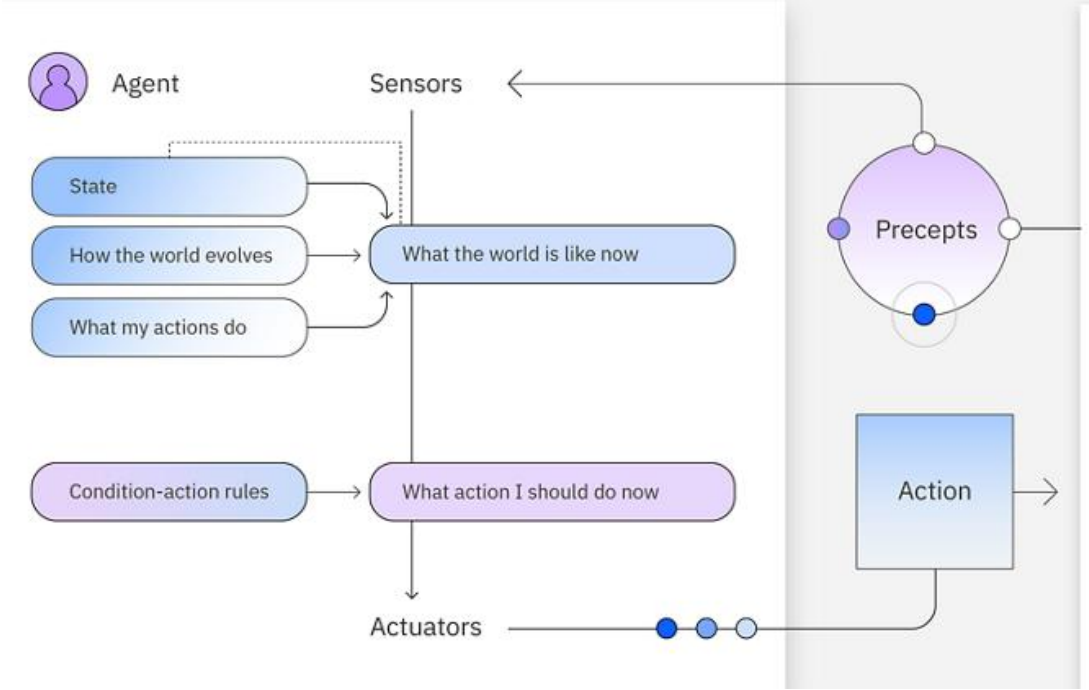
Not feasible for dynamic scenarios that require memory, learning or long-term planning



Model-based reflex agents

Relies on condition-action rules but also incorporates internal model that tracks current state of environment and understands how past interactions impacted it

Example: robot navigating a room reacts to obstacles in its path but also remembers obstacles already encountered.



Function effectively in partially observable environments, as they can handle situations with context to be remembered and used for future decisions

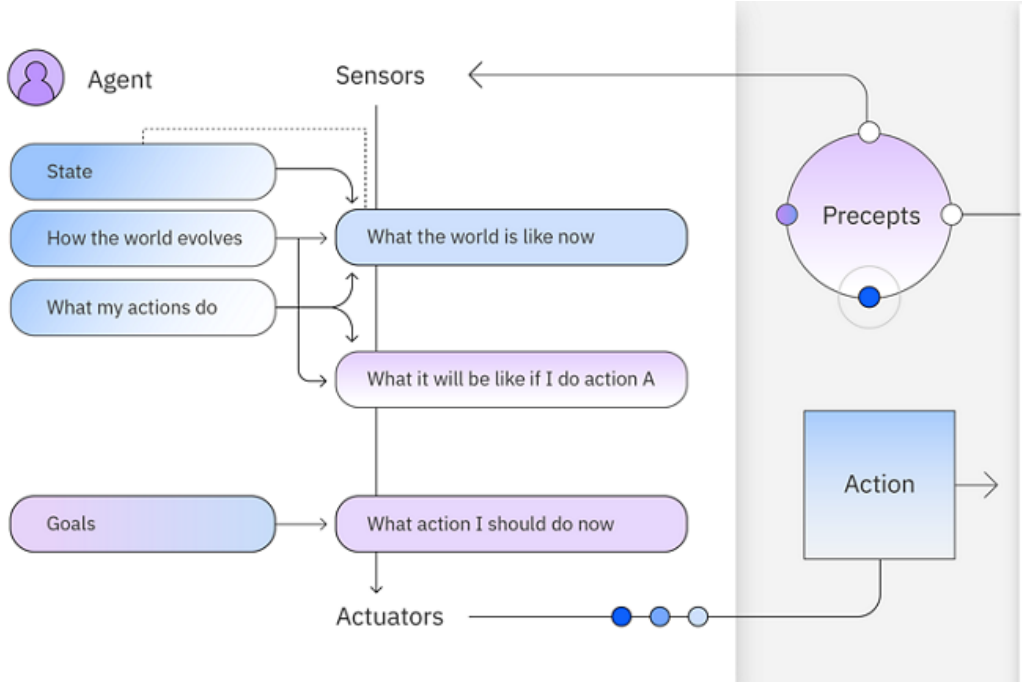
Goal-based agents

Incorporates proactive, goal-oriented approach to problem-solving

Considers overall objectives and uses planning / reasoning to choose actions

Example: robot navigates a building to reach specific room: path to minimize detours and avoids known obstacles

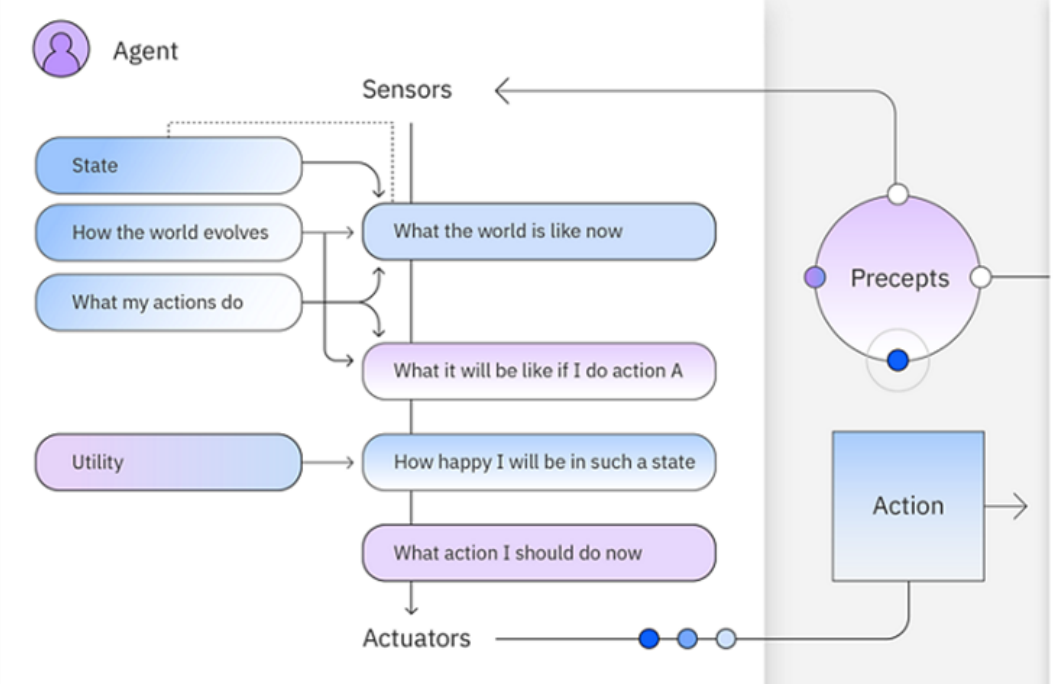
Goal-based agents rely on preprogrammed strategies or decision trees for evaluating objectives



Utility-based agents

Evaluates, selects actions to maximize overall benefit. Not specific objective, but range of possible outcomes

- Assign utility value to each outcome to determine optimal course of action
- Nuanced decision-making in situations with multiple goals or tradeoffs are involved.



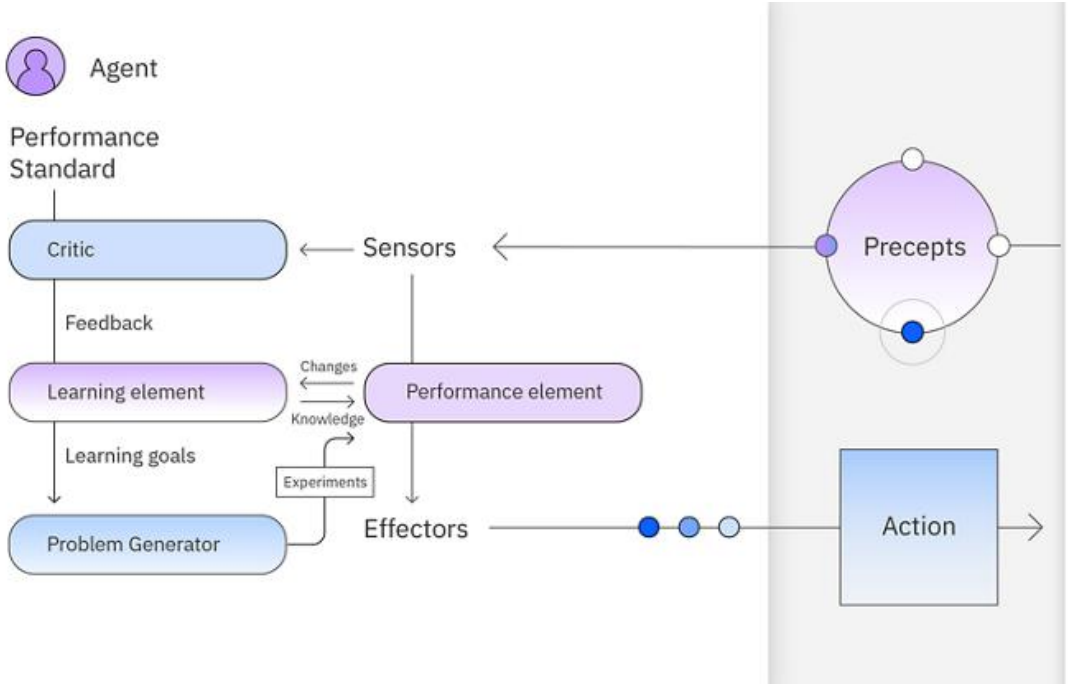
Example: self-driving car selects between speed, fuel efficiency, safety when navigating a route

Effective to balance competing objectives, adapt to changing conditions, ensuring intelligent, flexible behavior

Learning agents

Improves performance over time by adapting to new experiences and data based on feedback from environment. Main components:

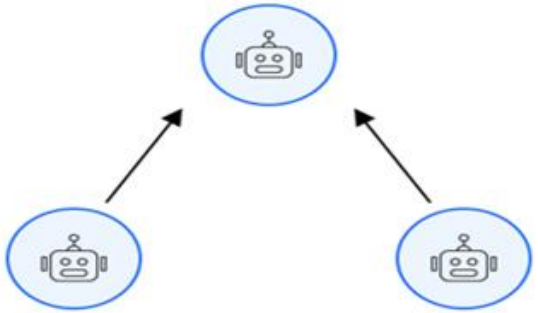
- Performance: Determines actions based on knowledge base
- Learning: Adjusts and improves the agent's knowledge based on feedback and experience.
- Critic: Evaluates agent's actions and provides feedback as reward or penalty
- Problem generator: Suggests exploratory actions to discover new strategies and improve its learning.

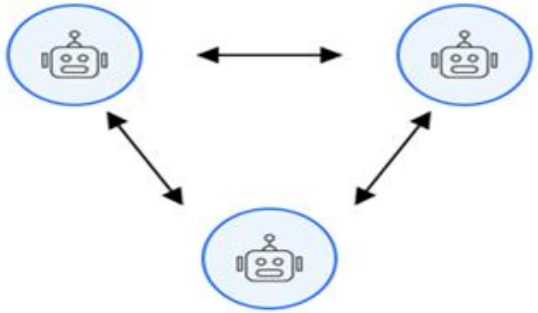


Example: Reinforcement learning

Multi-agent systems (Agentic AI)

Hierarchical agents designed to break down complex problems into smaller, manageable subtasks to handle complex problems in real-world scenarios.





Higher-level agents focus on overarching goals, lower-level agents handle specific tasks

AI orchestration: integrates different types of AI agents into intelligent and adaptive multi-agent system to operate in real time, responding to dynamic environments, improving performance based on past experiences.

Model Context Protocol (MCP)

MCP = open standard created by Anthropic (2024) that defines a universal way for AI models to connect to external tools, data sources, and services.

Why is MCP needed

- LLMs are Isolated by Design: trained on static data, no built-in access to live systems, APIs, or private databases
- N × M Integration: Each model-tool pair requires a custom connector => N models × M tools = N·M integrations to maintain
- Context Window ≠ Real World: RAG patches the symptom; it does not give the model agency to act on live, mutable systems.

MCP: Basics

1. MCP is to AI agents what HTTP is to the web — a shared transport contract.
2. Decouples model from infrastructure: swap LLMs or tools independently.
3. Open standard (MIT licence) with multi-vendor adoption
4. Security model: servers are sandboxed; hosts control capability grants
5. Designed for agentic loops: streaming, long-running tasks, human-in-loop

MCP Architecture: Three-Layer Model

Host

- Orchestrates the overall AI workflow
- Runs the LLM & manages conversation state
- Creates and manages MCP-Client instances
- Enforces user permissions & security policies

e.g. Claude Desktop, IDE plugin, Chat UI

JSON-RPC 2.0

Client

- Maintains a 1-to-1 session with one Server
- Handles capability negotiation on connect
- Forwards tool/resource calls from the LLM
- Routes responses back to the Host

e.g. MCP SDK inside your application

JSON-RPC 2.0

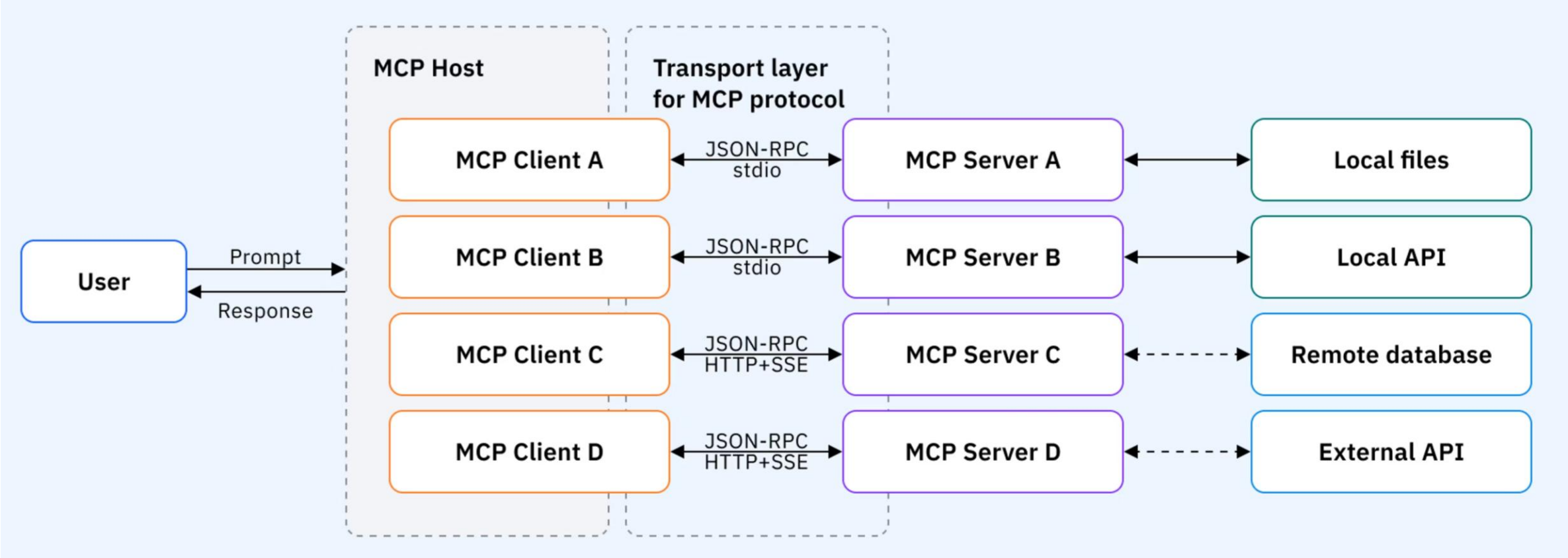
Server

- Exposes Tools, Resources & Prompts
- Implements domain logic (DB, APIs, shell…)
- Stateless or stateful — implementation choice
- Runs locally or as a remote HTTP service

e.g. GitHub MCP, Postgres MCP, filesystem

Transport: stdio (local subprocess) or HTTP + Server-Sent Events (remote service)

MCP Architecture



MCP Architecture: Three-Layer Model

Tools	Resources	Prompts
Model-invoked actions Functions the LLM calls with structured JSON arguments. The server executes them and returns results to the model. Typical uses: DB queries, file writes, REST API calls, shell commands, email dispatch.	Application-controlled read-only data Data sources the host surfaces to the model: files, web pages, DB rows, images, blobs. Addressed by URI scheme — file://, postgres://, https:// — making them universally identifiable. The model can read but not modify.	User-triggered reusable templates Pre-built prompt templates with typed arguments. Users select them (like slash commands); the server fills in context and returns a ready-to-run message sequence. Standardized workflows: code review, summaries, incident reports.

How does the MCP work?

MCP allows LLM to request help from external tools to answer a query/complete a task. Steps for example e-mail a DB-generated report

1. Request and tool discovery: LLM cannot access a database or send emails => uses MCP client to search for available tools
2. Tool invocation: LLM generates structured request to use the tools, e.g. it calls database_query tool, specifies the name, sends the request to MCP server.
3. External action and data return: MCP server receives and translates request into a secure SQL query, retrieves report, formats data, and sends back to LLM.
4. Response generation: LLM calls the email_sender tool providing the email address and report, MCP server confirms the action was completed.
5. Final confirmation LLM to user

MCP vs RAG?

Feature	Model Context Protocol (MCP)	Retrieval-Augmented Generation (RAG)
Primary goal	2-way communication for LLMs to access and *interact* with external tools, data sources, services	Enhance LLM by *retrieving information* from knowledge base *before* generating a response.
Mechanism	Standardized protocol for LLMs to *invoke external functions* or request data from specialized servers, enabling actions and dynamic context integration.	Incorporates retrieval component to pull information from a knowledge base or data source and augment the LLM prompt.
Output type	Enables LLMs to *generate structured calls* for tools, receive results, and generate human-readable text	LLMs generate responses based on training data *augmented* by relevant external documents.
Interaction	Designed for *active interaction* and execution of tasks in external systems	Primarily for *passive retrieval* of information to inform text generation
Standards	Open standard	Technique / framework, but not universal protocol
Use cases	AI agents performing tasks fetching real-time data, advanced integrations.	Question-answering systems, chatbots providing up-to-date factual information,

MCP: Ecosystem

Hosts: Claude Desktop, VS Code, Cursor, Zed

Servers: GitHub, Postgres, Filesystem, Brave Search, Slack, Docker…

SDKs: Python, TypeScript, Java, Kotlin, C#

Adopted by: Anthropic, OpenAI, Google DeepMind, Microsoft, and others

Agent-to-Agent protocol (A2A)

Open communication standard for AI agents introduced by Google in 2025, donated to Linux Foundation as open-source project

- Designed for multi-agent systems, enabling interoperability between AI agents from different providers or built on different frameworks
- Addressing missing integrations, security gaps, and no agent protocols

How it relates to MCP

- MCP is for AI agents communicating with external services — APIs, databases, tools.
- A2A focuses on agent-to-agent collaboration
- Both protocols complement each other

A2A core concepts

Components

1. Agent Cards: advertise capabilities in JSON, allow agents to discover best agent for task
2. Task management: defined lifecycle, can be short or long-running
3. Collaboration: Agents exchange messages with context, replies, artifacts, user instructions.
4. UX negotiation: Messages include "parts" (fully formed content like images) so agents can adapt to different content modalities.

A2A is built on existing web standards — HTTP, SSE (Server-Sent Events), JSON-RPC

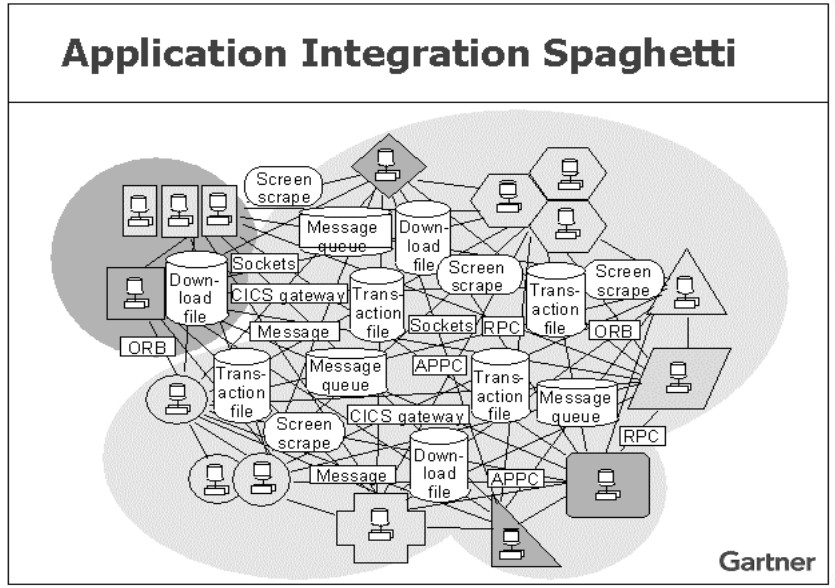
Supporting organizations include Google, Microsoft, AWS, Adobe, Salesforce, ServiceNow, SAP

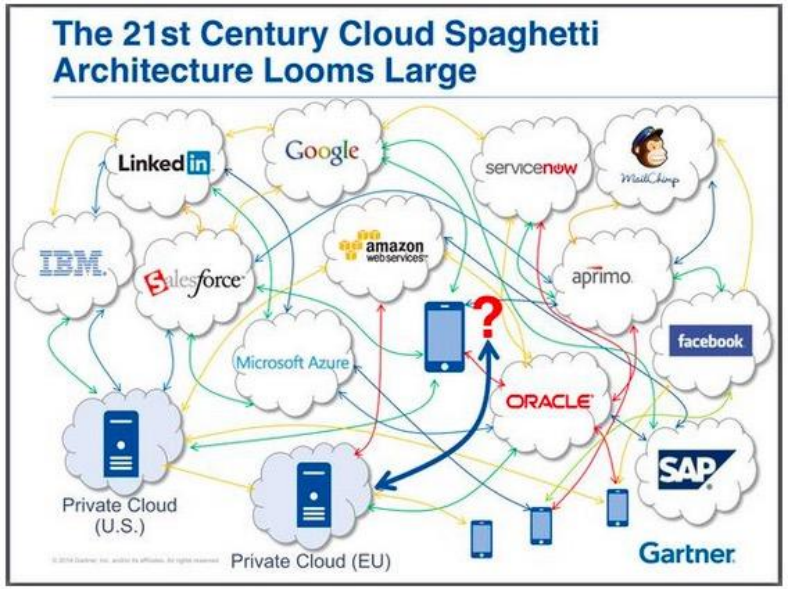
Programming Models

4.5 (Patterns for) Enterprise Application Integration

Enterprise Application Integration (EAI) …

"… is the task of making disparate applications work together to produce a unified set of functionality"





Hohpe, Woolf: Enterprise Integration Patterns – Designing, Building, and Deploying Messaging Solutions

@Gartner

EAI (Enterprise Application Integration)

Integration framework composed of a collection of technologies and services which form a middleware to enable integration of systems and applications across an enterprise.

Many types of business software (supply chain management applications, ERP, CRM etc.) typically cannot directly communicate with one another to share data or business rules

Business owners want to avoid vendor lock-in → take the best software for the purpose and integrate with the existing infrastructure

EAI = process of linking such applications within a single organization to simplify and automate business processes to the greatest extent possible, while at the same time avoiding having to make sweeping changes to the existing applications or data structures.

EAI - Motivation

Enterprises often comprised of several (hundreds, thousands, …) applications

- Applications are custom built, developed by third party, part of legacy systems
- Defining a clear separation between the applications is difficult → business processes often span multiple systems

How can this be possible? Management issue?

- Developing single application that suit the needs of entire business is almost impossible
- Using several applications → flexibility to choose the most suitable application, instead of choosing one big solution that performs well but not optimal for each use case
- Businesses evolve → Applications are continuously adapted, new applications introduced

EAI - Challenges

EAI deals with integration of multiple applications, running on multiple platforms in different locations

- Communication between business units and IT departments:
  - Remember initial DS motivation (5): modeling of real situations
  - IT systems often reflect the organizational structure of businesses → separated into functional areas/units
  - Communication between units is even more challenging due to heterogeneity
  - EAI needs to break this separation → (IT) departments no longer control specific applications since each applications is part of an overall flow of information and services

EAI - Challenges

EAI deals with the integration of multiple applications, running on multiple platforms in different locations

- Communication between business units and IT departments
- Integration solutions often have major impact on the busines
  - Once core applications are integrated, the proper functioning of the integration solutions becomes critical to the business
  - Failures or misbehaving integrations can cause severe damage to a business

EAI - Challenges

EAI deals with the integration of multiple applications, running on multiple platforms in different locations

- Communication between business units and IT departments
- Integration solutions often have major impact on the business
- Integration developers have no full access to the participating applications
  - Legacy systems or external applications cannot be changed
  - How to enable a legacy application for a messaging scenario that was not designed to send and receive messages?

EAI - Challenges

EAI deals with the integration of multiple applications, running on multiple platforms in different locations

- Communication between business units and IT departments
- Integration solutions often have major impact on the business
- Integration developers have no full access to the participating applications
- Interoperability & standards: A common representation of data does not imply common semantics
  - Consider XML, which acts as a standardized, self-descriptive external data representation
  - Consider the term “account”, which can have different semantics in different applications
  - Each application may make different assumptions or constraints regarding the presentation of an account

EAI - Challenges

EAI deals with the integration of multiple applications, running on multiple platforms in different locations

- Communication between business units and IT departments
- Integration solutions often have major impact on the business
- Integration developers have no full access to the participating applications
- Interoperability & standards: A common representation of data does not imply common semantics
- Developing EAI is challenging, operating and maintaining can be even more challenging
  - EAI deals with many different technologies and incorporates many distributed resources
  - Deployment, monitoring and troubleshooting become complex tasks

EAI – Loose Coupling

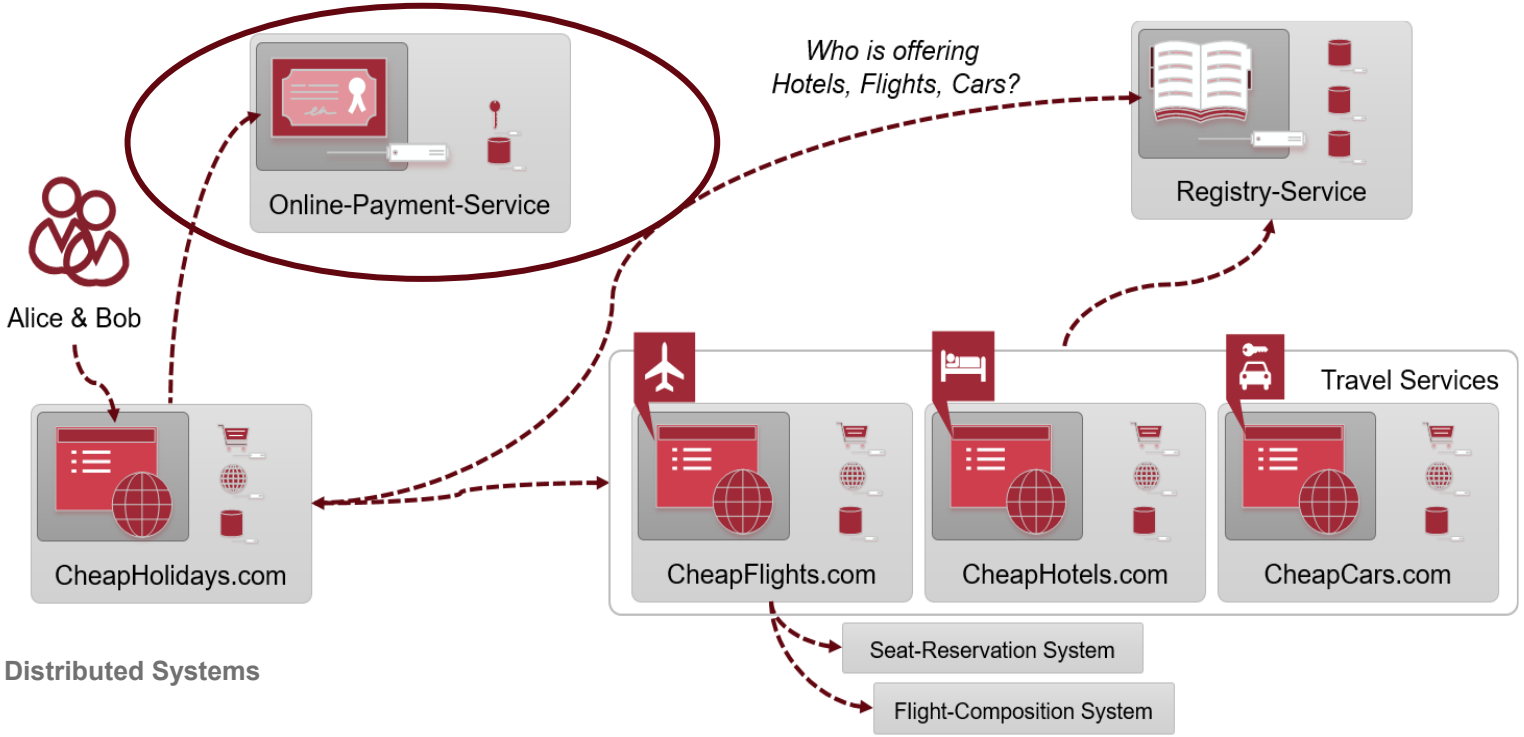
Loose coupling is about reducing the assumptions the systems make about each other when exchanging information

Common trade-off

- More assumptions (protocol, data-format, …) → more efficient communication
- Less assumptions → more tolerant to changes in the systems
- System will change/adapt over time → Loose coupling important requirement for EAI

EAI – Loose Coupling

Loose coupling example: Payment front-end (e.g. web application) needs to be integrated with the financial backend to provide the service

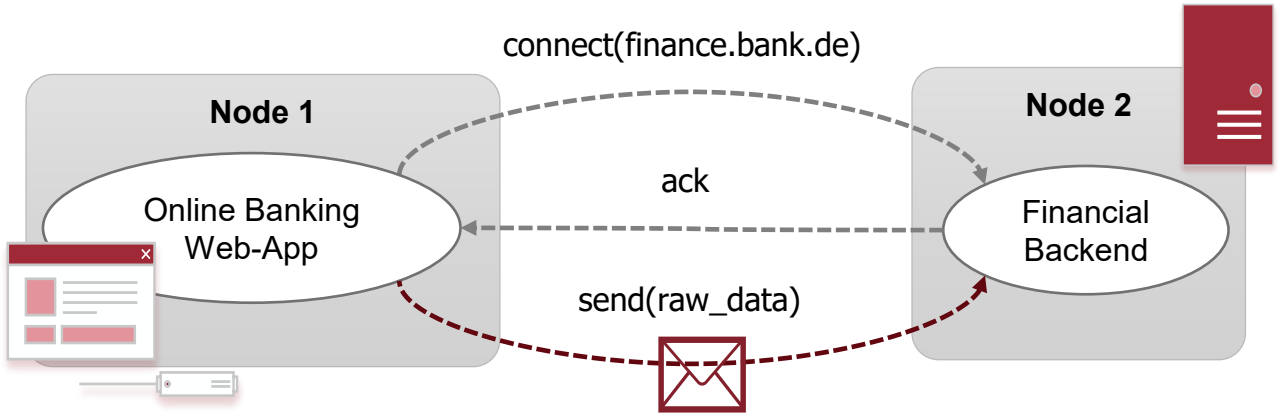


EAI – Loose Coupling

Simple approach: use TCP/IP and send a message to deposit/withdraw money

⇒ Can be implemented with about 20 lines of code (assuming no exception handling)

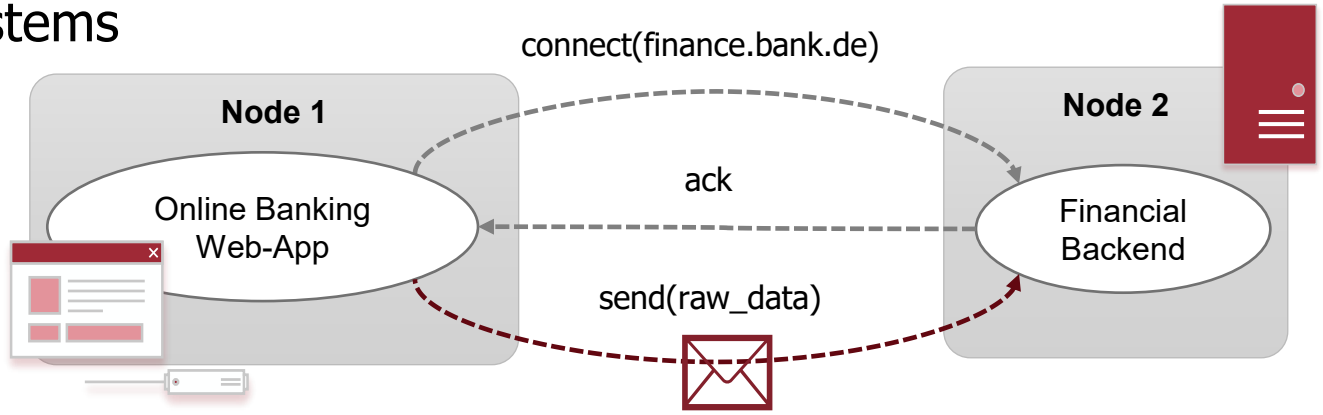
⇒ Fast and easy to implement and test → where is the problem?



EAI – Loose Coupling

Following assumptions are made → applications are tightly coupled

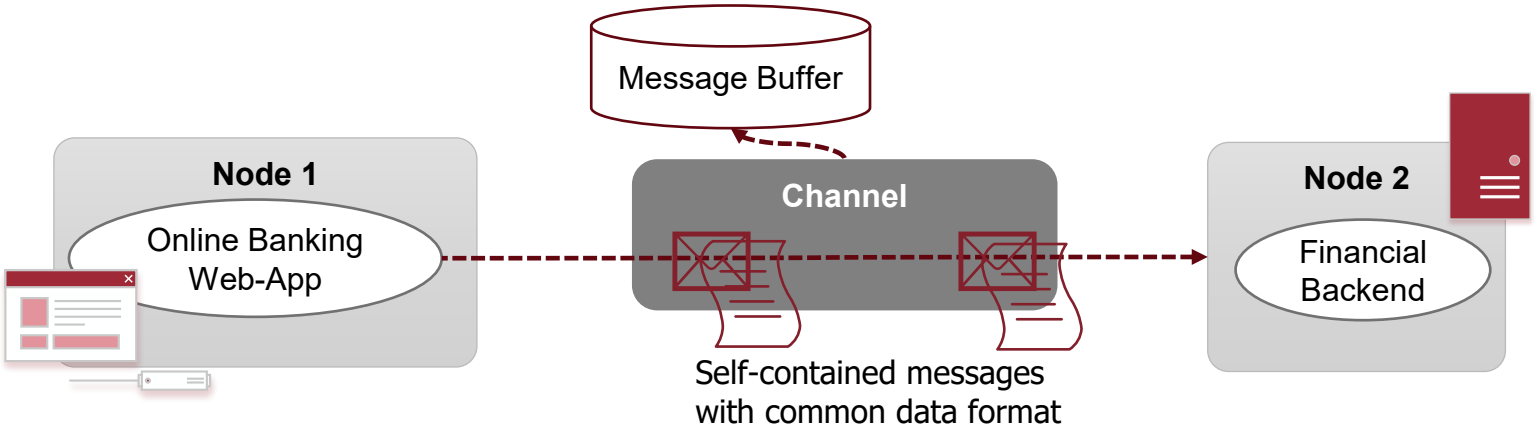
- Heterogeneity of platforms: internal representation of numbers and values must match (e.g. Big vs. Little Endian)
- Location: address of the financial backend is hard-coded (Space decoupling)
- Time: all components must be available at the same time (Time decoupling)
- Data format: parameters and parameter types must match, cannot insert additional ones without changing both systems



EAI – Loose Coupling

How to reduce the assumptions?

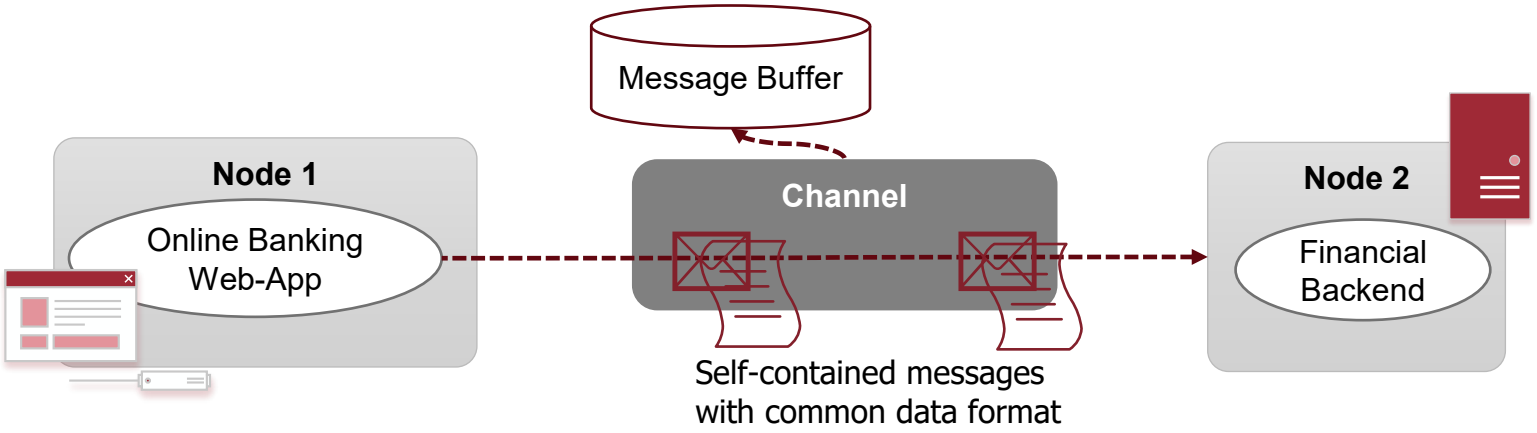
- Use a standard, self-descriptive data format such as XML / JSON
- Send data to addressable channel: Wrap raw data into messages → channel can buffer messages (e.g. a queue)
- Enhance channel with format transformations: Applications can be changed independently



EAI – Loose Coupling

The new solution is much more tolerant to change, but introduces complexity

- Cannot be implemented with 20 lines of code anymore
- Message-oriented middleware (e.g. queueing systems) can help to reduce complexity
- What about more complex integration scenarios? How to design mission critical EAIs?



EAI - Patterns

Why do some people write better code/find better solutions than others do?

It's largely about experience:

- If one have solved many problems, one can often relate new problems and identify "patterns" of problems and related solutions
- Patterns are best practices to common types of problems, evolved over time by trial-and-error
- Patterns often related to other patterns and need to be combined to solve complex problems

Patterns are one of the most important concepts when talking about software design:

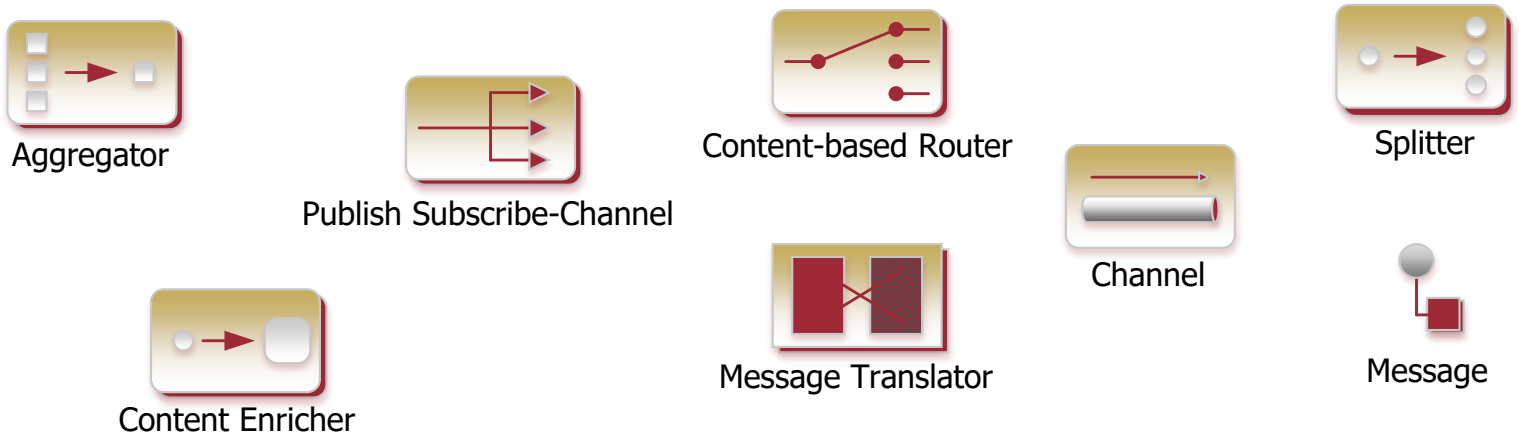
- Examples: Observer, Model View Controller, Lazy Load, Singleton, Iterator, Factory Method
- Gang of Four book: Gamma et al.: Design Patterns. Elements of Reusable Object-Oriented SW

Nowadays: ask ChatGPT about the best pattern

EAI - Patterns

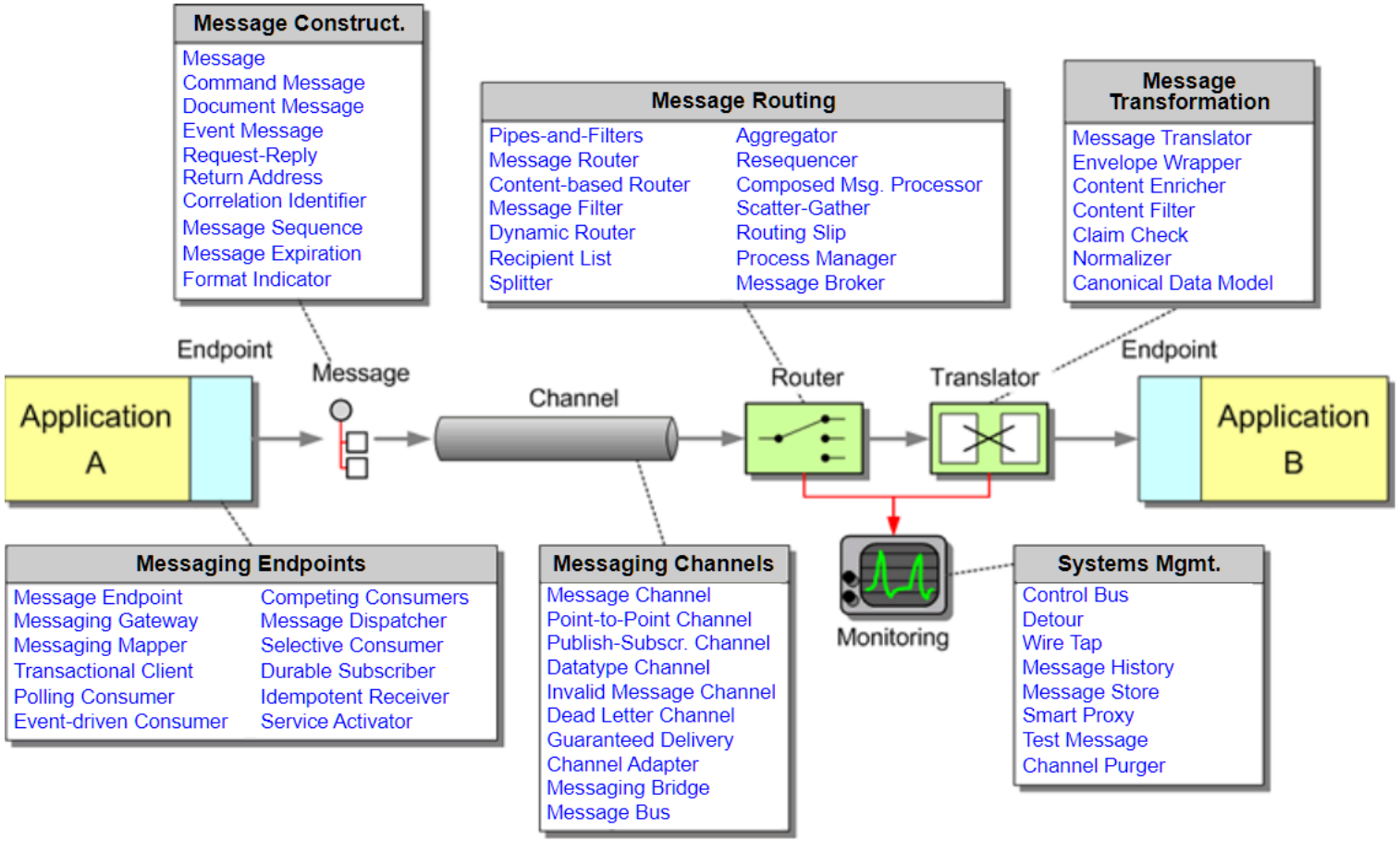
A set of EAI patterns that help us solve integration problems exist

The following slides present EAI patterns and an integration example as described in [1]



[1]: Gregor Hohpe, Bobby Woolf: Enterprise Integration Patterns – Designing, Building, and Deploying Messaging Solutions; Addison-Wesley (2003)

Example



https://camel.apache.org/

EAI – Patterns – Fundamental Approaches

Several approaches for application integration evolved over time

- File Transfer: applications exchange data by reading and writing files
- Shared Database: application store shared data in a shared database
- Remote Procedure Invocation: applications expose procedures using a well-defined interface, exposed procedures can be called remotely
- Messaging: each application is connected to a messaging system (e.g. a queuing middleware), messages are used to exchange data, notify about events, and trigger operations

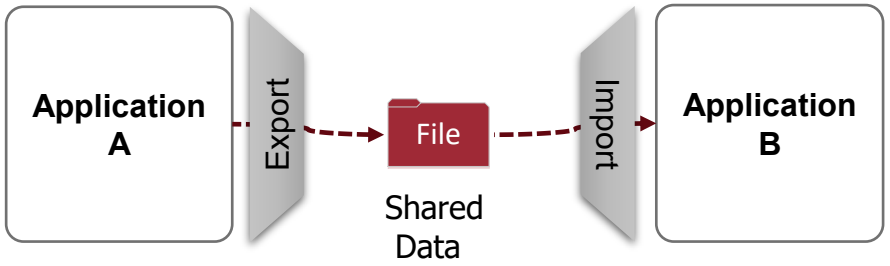
EAI – Patterns – File Transfer

Context: multiple applications built using different languages and platforms

Problem: How to integrate multiple applications to work together and exchange information?

Solution:

- Files as universal storage mechanism available on every platform/language
- Applications produce files that contain information other applications must consume
- Integrators take care of properly transforming files (different formats)
- Files are created at regular intervals according to business needs



EAI – Patterns – File Transfer

Discussion:

- Files become public interfaces of the application
- No extra tools/middleware required for integration, but developers do a lot of work themselves
- Agree on file names and directories, locking to prevent reading while writing, file transfer, who deletes old files?
- When to produce and consume files? Using File Transfer, updates typically occur infrequent (files generated every hour, every day, …)
- Data inconsistencies among the integrated systems are likely to occur

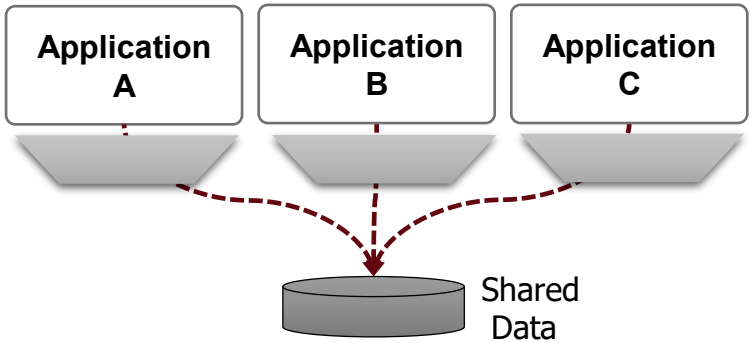
Relation to other patterns:

- To make data available more quickly and enforce common data formats, use Shared Database
- To integrate the functionality of applications rather than data, use Remote Procedure Invocation
- To enable for fine-grained and frequent exchanges of small amounts of data, use Messaging

EAI – Patterns – Shared Database

Context: multiple applications built using different languages and platforms, information needs to be shared rapidly and consistently

Problem: How to integrate multiple applications to collaborate and exchange information?



Solution:

- Applications integrated by storing data in a single shared DB
- A common data schema is defined by the database to handle all the applications

EAI – Patterns – Shared Database

Discussion:

- Applications share a consistent view on data if they all access same database
- Almost all platforms support SQL and can be integrated with a shared database
- No transformations are required due to common data schema, but how to define this?
  - Universal/generic schemas can become very abstract and difficult to work with
  - Critical systems may not accept delay and complexity introduced by common schema
  - What about schema updates only by certain applications, what about legacy/external applications with no support for the schema?
- Database can become performance bottleneck (especially with web-scale applications)

Relation to other patterns:

- If loose coupling more important than timeliness and common data format, use File Transfer
- To integrate the functionality of applications rather than data, use RMI
- To enable for fine-grained and frequent exchanges of small amounts of data, use Messaging

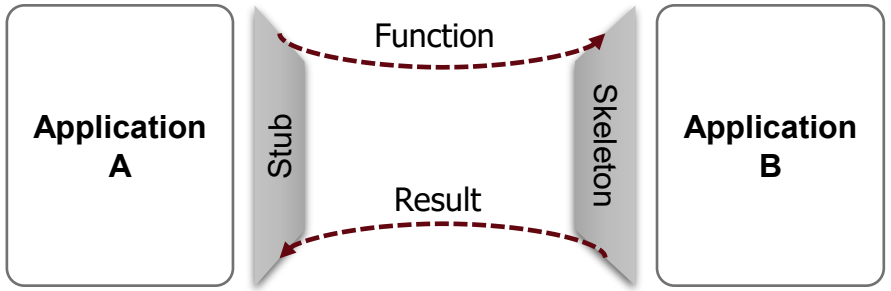
EAI – Patterns – Remote Method Invocation

Context: multiple applications built using different languages and platforms, information and processes need to be shared.

Problem: How to integrate multiple applications so that they work together and can exchange information?

Solution:

- Each application is developed as an object or component with encapsulated data
- An interface is provided to interact with the application
- Not only propagate changes in data, but additionally trigger required processes



EAI – Patterns – Remote Method Invocation

Discussion:

- Applies the principle of encapsulation: if data owned by another application is required, ask for it. If data owned by another application must be changed, make a call
- Interfaces encapsulate data → each application can change actual formats independently: Multiple interfaces to internal data can be provided → different views
- Interfaces must be negotiated between collaborating pairs of applications (tight coupling)
- Remote Procedure Calls have same semantics but behave different than local calls: If not considered by developer, system can become slow and unreliable

Relation to other patterns:

- To integrate multiple applications more loosely coupled and asynchronous, use Messaging

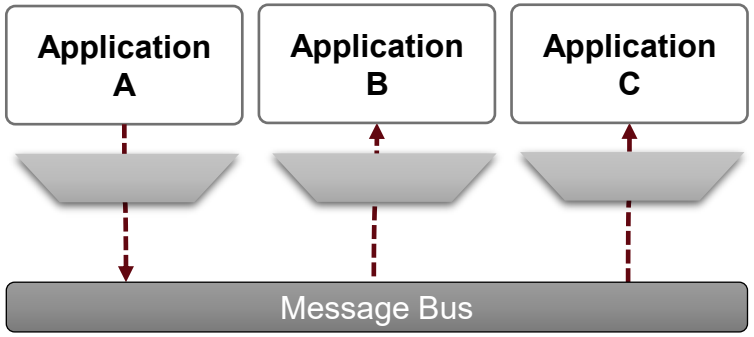
EAI – Patterns – Messaging

Context: multiple applications built using different languages and platforms, information and processes need to be shared.

Problem: How to integrate multiple applications so that they work together and can exchange information?

Solution:

- Transfer packets of data frequently, immediately, reliably
- Enable for asynchronous and loosely coupled operation
- Use customizable data formats



EAI – Patterns – Messaging

Discussion:

- Asynchronous messaging: while in transit, messages can be transformed without sender or receiver being aware
- Many-to-One, One-to-Many and other topologies are supported (group communication)
- Data can be exchanged, behavior/processes can be triggered
- Similar to File Transfer, inconsistencies can occur (even using very frequent exchanges)
- Much more decoupled than RMI, but a lot of transformation work necessary
- Fundamental aspects that need to be considered when using Messaging:
  - How to transfer packets of data?
  - How to figure out where to send the data?
  - How to know what data format to use?
  - How to connect the application to the messaging system?

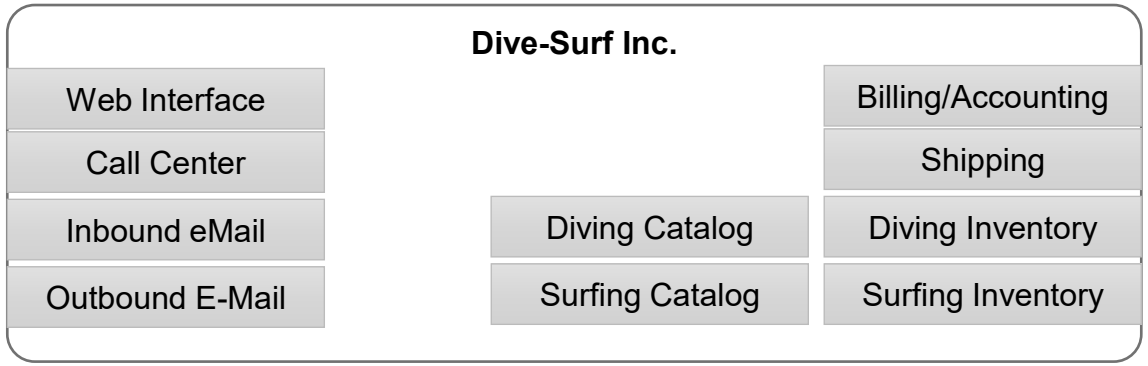
Self-Study

Messaging example using different pattern

EAI – Patterns – Messaging Example

Before Alice and Bob start their holidays, they need equipment

- Alice prefers to go diving, Bob prefers to go surfing
- Dive-Surf Inc. is a retailer selling diving and surfing equipment to customers
  - Dive-Surf Inc. is a company with a variety of custom applications: Billing, Shipping, Customer Interfaces
  - Dive-Surf Inc. started selling surfing equipment and extended their business later by selling diving equipment as well → as a result, two inventory and catalog systems exist



EAI – Patterns – Messaging Example

Taking Order - three channels exist to place orders: Web interface, call center, eMail

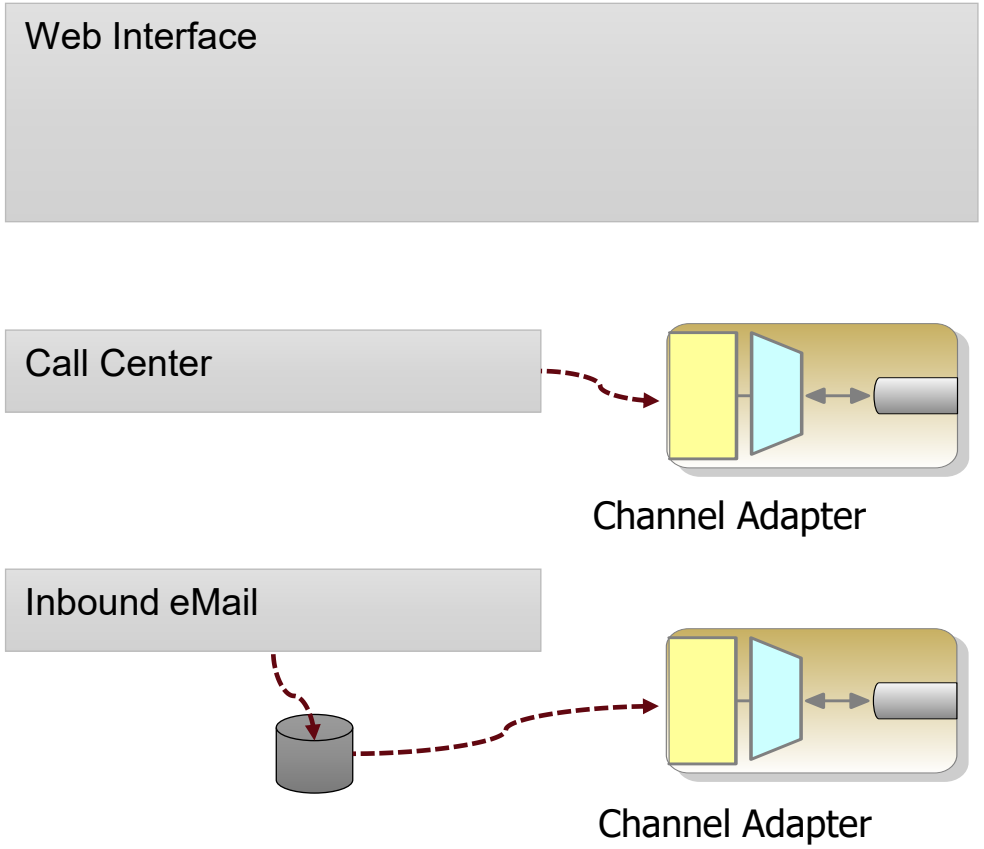
- Resulting processes require manual interaction and need to be integrated to safe costs
- Each system based on a different technology and uses different formats to store orders
  - The call center is a packaged desktop application
  - The web interface is a custom J2EE application
  - The eMail requires manual entry of orders into a small Microsoft Access application

Goal: treat all orders equally regardless of source → customers should place orders using one channel (web interface) and check status using another one (call center)

EAI – Patterns – Messaging Example

Taking Order - three channels exist to place orders: Web interface, call center, eMail

- Call center and eMail channels are packaged applications not designed to be integrated
- Channel Adapters to connect to the messaging system



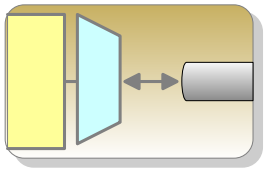
EAI – Patterns – Channel Adapter

Context: messaging is commonly used to integrate multiple, disparate applications

Problem: How to connect an application to the messaging system so that it can send and receive messages?

Solution:

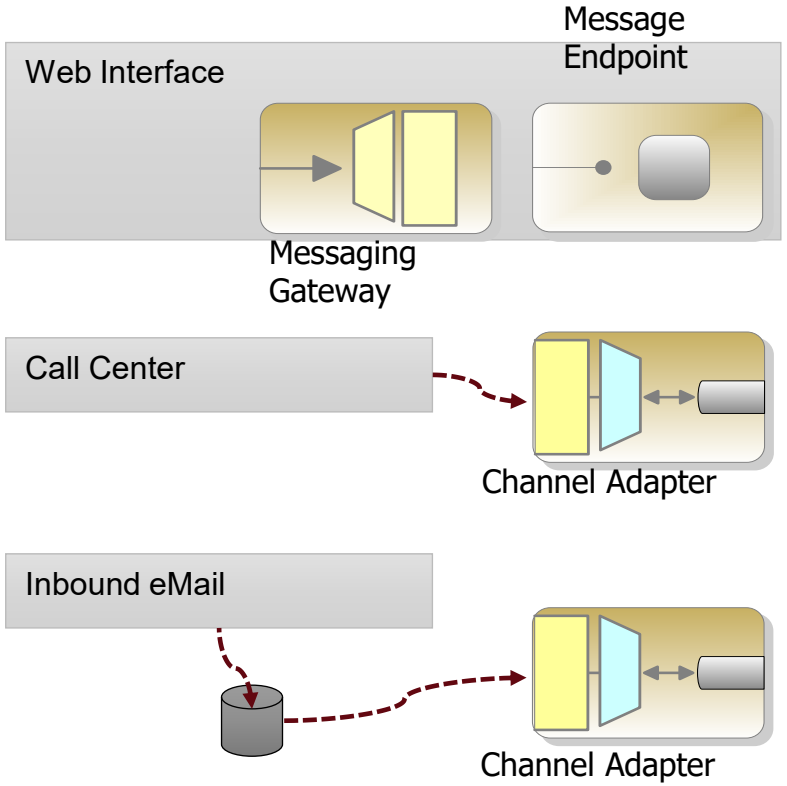
- Channel Adapter can access the applications API or data to publish messages on a channel
- Channel Adapter can receive messages from channel and invoke functionality of application
- The adapter can connect to user interface, business logic (API) or database of the application
- User Interface: if no API is available or application runs on platform that cannot be accessed → parse output of user interface and also generate input for it
- Adapter can run on different node than application
- Adapters often require a corresponding Message Translator



EAI – Patterns – Messaging Example

Taking Order - three channels exist to place orders: Web interface, call center, eMail

- Web interface is custom build → implement a Message Endpoint directly
- Messaging Gateway can be used to isolate application from messaging logic



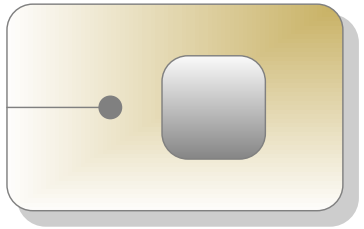
EAI – Patterns – Message Endpoint

Context: application communicate by sending messages through Message Channels

Problem: How does an application connect to a message channel to send and receive Messages?

Solution: Message Endpoint connects an application to a channel

- The endpoint acts as a client to the messaging system that can be used by the application to send and receive messages
- Endpoints are channel specific → multiple endpoints required if multiple channels are used
- Message Endpoint is a specialized Channel Adapter, directly embedded into the application
- Message Endpoint should be designed as a Messaging Gateway



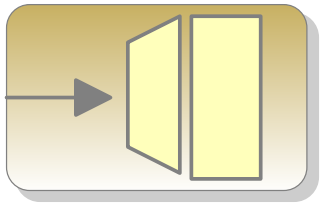
EAI – Patterns – Messaging Gateway

Context: an application accesses another system through Messaging

Problem: How to encapsulate access to messaging system from rest of the application?

Solution: Messaging Gateway wraps method specific method calls and exposes a domain-specific set of methods to the application

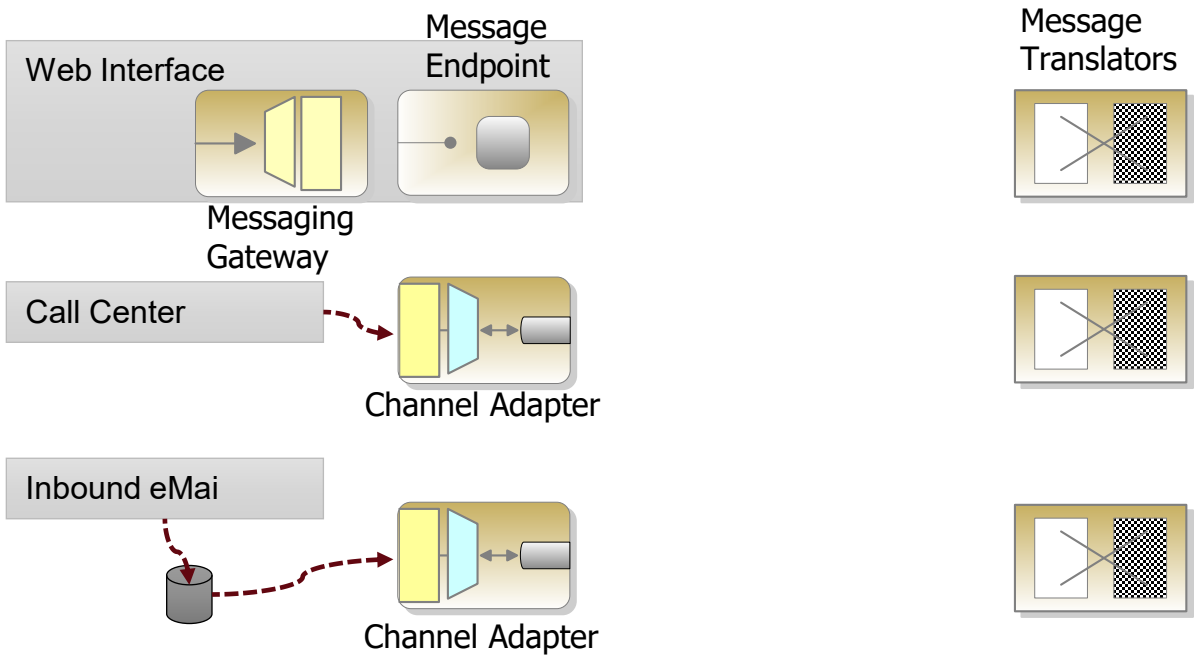
- placeOrder(Customer cust, Order ord) instead sendMessage(new OrderMsg(cust, ord))
- Only the Messaging Gateway knows about the messaging system → makes it easy to change messaging system
- Can operate synchronous (blocking) or asynchronous (event-driven using callbacks)



EAI – Patterns – Messaging Example

Taking Order - three channels exist to place orders: Web interface, call center, eMail

– Message Translators, because each customer interface uses different data format



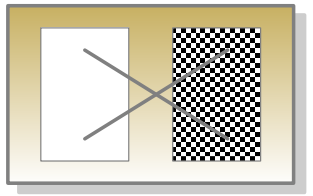
EAI – Patterns – Message Translator

Context: enterprise integration solutions route messages between various applications that often use proprietary data formats

Problem: Systems using different data formats communicate with each other using messaging?

Solution: Message Translator = filter between applications or other filters that translates one data format into another

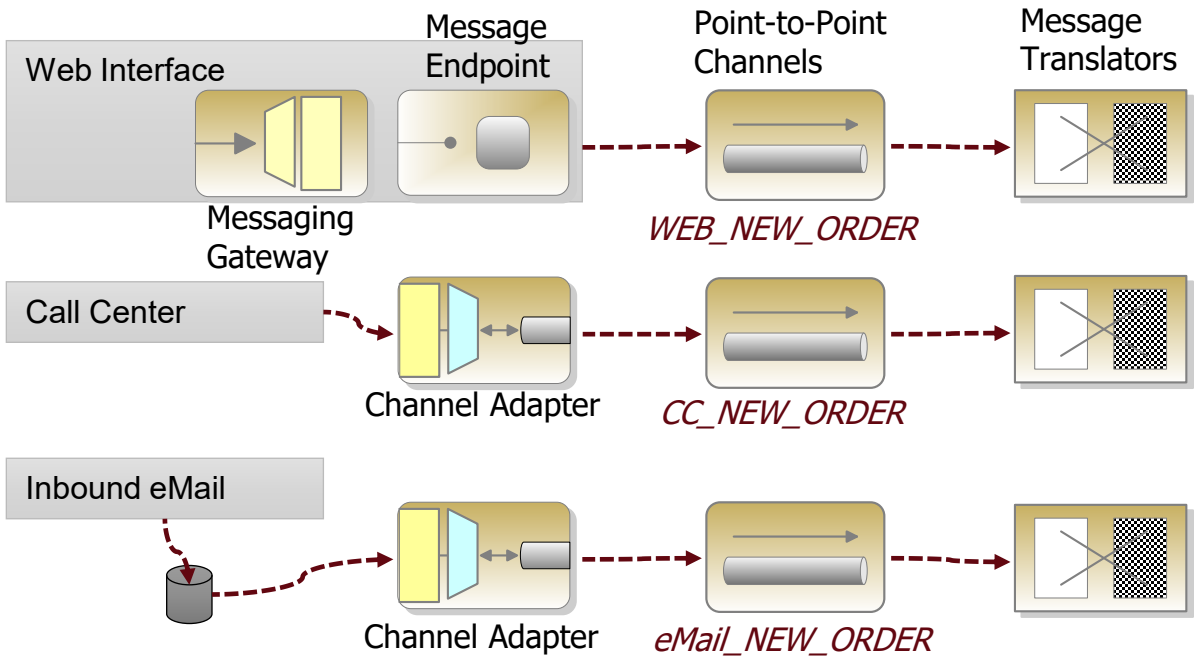
- Messaging equivalent of the Adapter pattern, message translators can be chained
- Levels of transformation:
  - Transport: use different protocols without changing content
  - Data Representation: parse data and generate new format (XML <→ binary, encrypted <→ decrypted)
  - Data Types: concerns field names and data types – e.g. convert phone numbers into international phone number strings, exchange name of country with country code
  - Data Structure: transform the data structures used – e.g. convert a relationship into an aggregation



EAI – Patterns – Messaging Example

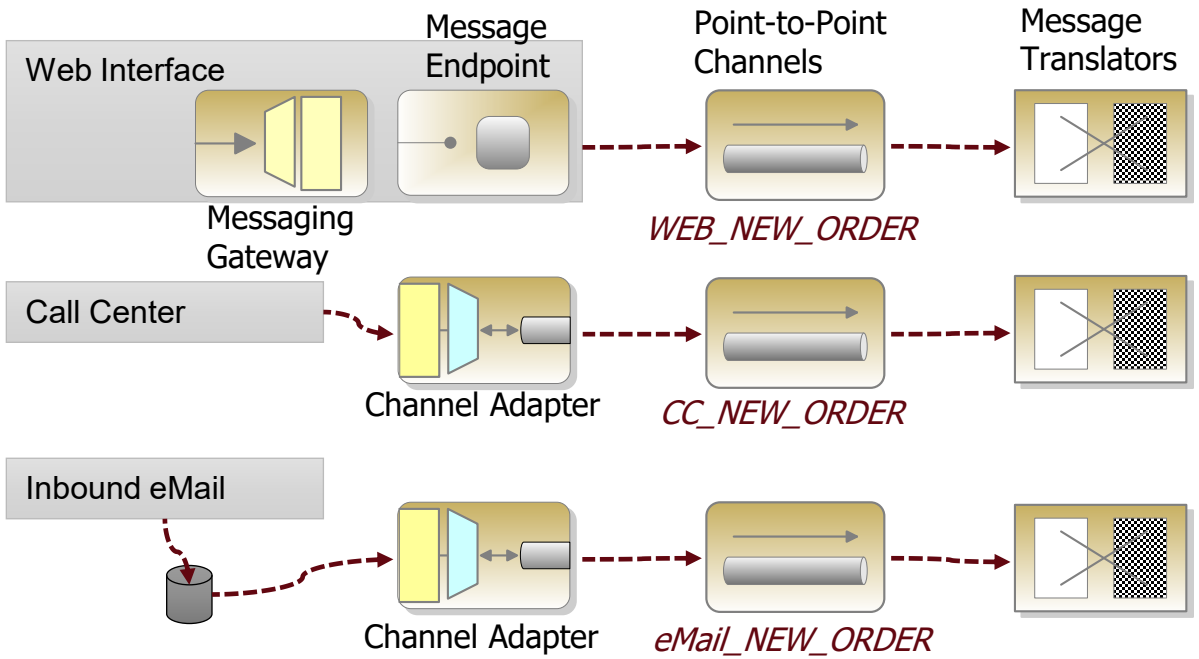
Taking Order - three channels exist to place orders: Web interface, call center, eMail

– Channel Adapters are connected to Message Translators using Point-to-Point channels



EAI – Patterns – Messaging Example

– Naming denotes that channels are application specific and should only be consumed by application and associated translator (e.g. WEB_NEW_ORDER)



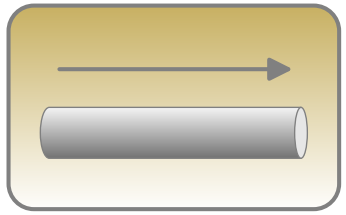
EAI – Patterns – Point-to-Point Channel

Context: application uses messaging to make remote procedure calls or transfer documents

Problem: How to ensure that exactly one receiver will receive the document or perform the call?

Solution: Point-to-Point Channel ensures that exactly one receiver will receive a particular message

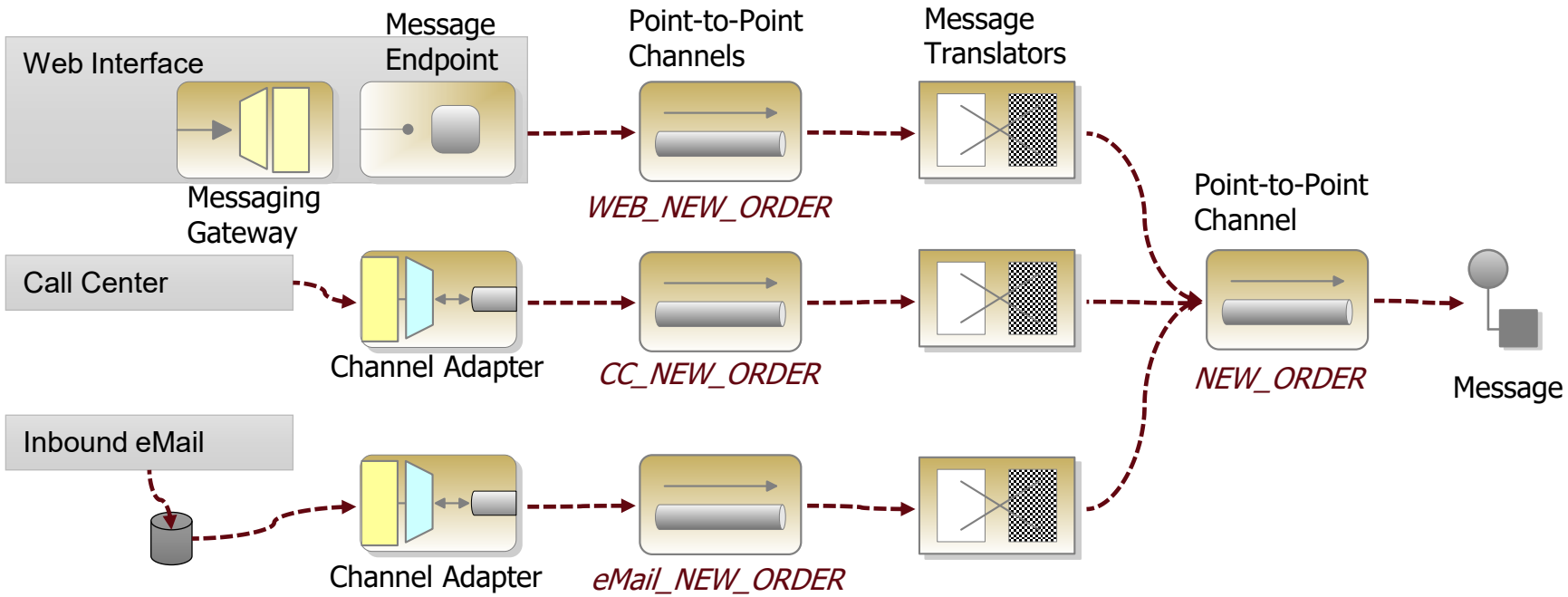
- The channel can have multiple consumers, but only one of them can successfully consume a particular message (e.g. a JMS Message Queue)



EAI – Patterns – Messaging Example

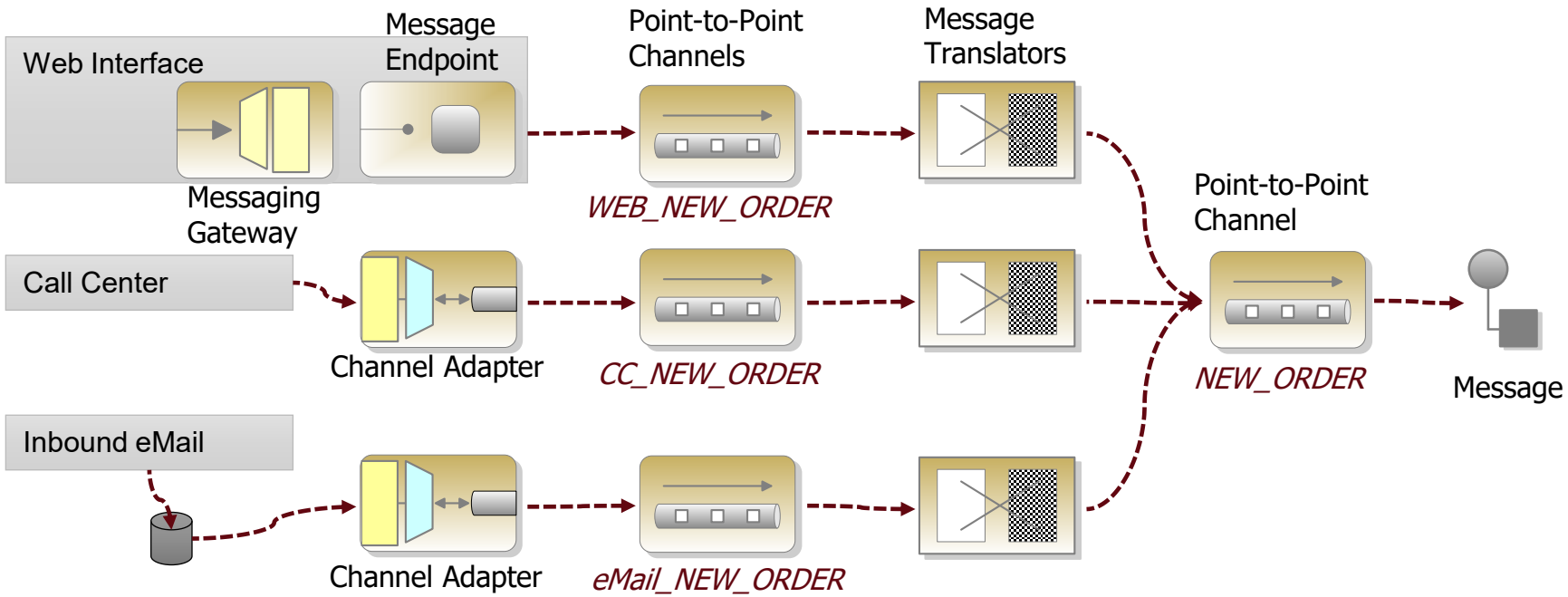
Taking Order - three channels exist to place orders: Web interface, call center, eMail

- Translators publish to NEW_ORDER channel, which allows to process the order independent of origin



EAI – Patterns – Messaging Example

– The channels comply to the Datatype Channel pattern, because only messages of one type are carried



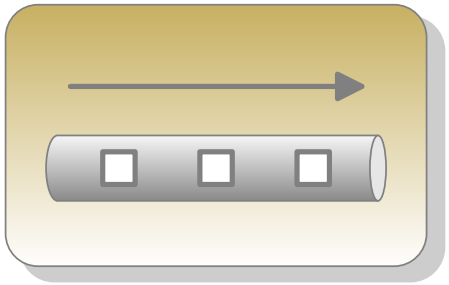
EAI – Patterns – Datatype Channel

Context: application uses messaging to transfer different types of data/documents

Problem: How to ensure that the receiver will know how to process the send data?

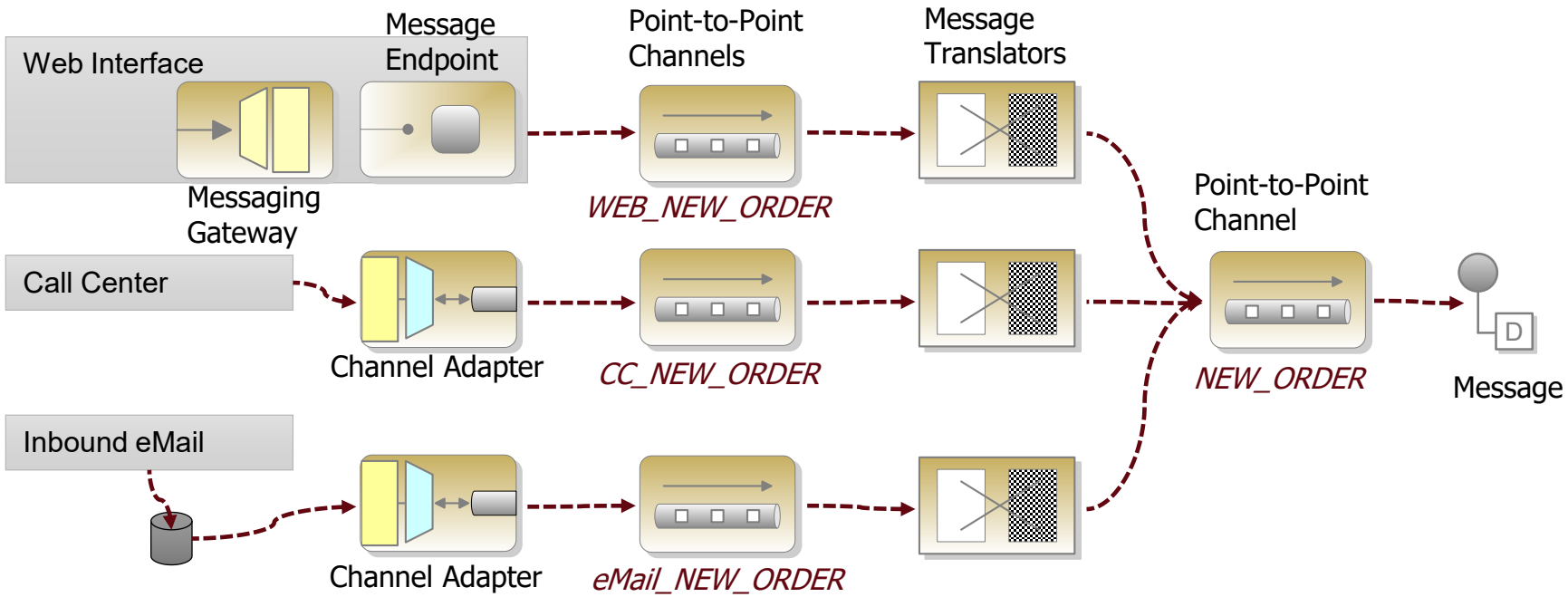
Solution: Datatype Channel enforces that all data on a particular channel is of the same type: Separate channels are used for each data type

- Trade-off: channels are cheap but not free. If creating a separate channel for each data type is not an option, Selective Consumers must be used to deal with channels carrying different data types
- If an existing application uses a single channel to publish several data types, but the integration scenario requires Datatype Channels, a Content-Based Router can be used to demultiplex the messages



EAI – Patterns – Messaging Example

– The resulting messages in the NEW_ORDER channel are Document Messages



EAI – Patterns – Document Message

Context: application transfers data to another one using messaging

Problem: How can messaging be used to transfer data between applications?

Solution: Document Message transfers a data structure between applications

- Compared to a Command Message, a Document Message just passes data and lets the receiver decide what to do with it (i.e. no explicit request to trigger an action)
- Document Messages are similar to Event Messages, but focus on the content
- Successfully transferring the document (Guaranteed Delivery) is important
- In contrast, event messages focus on Message Expiration and timing

EAI – Patterns – Messaging Example

After placing the order, it can be processed

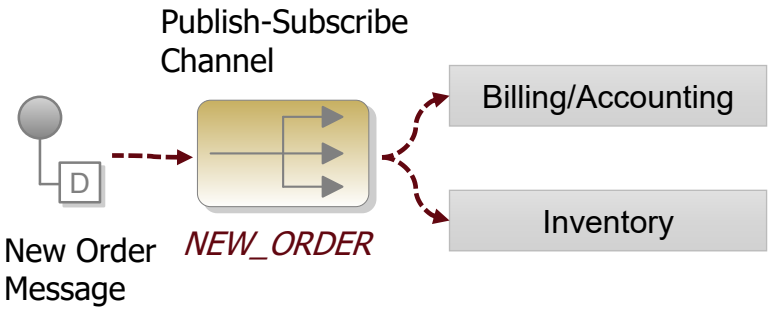
Processing Orders consists of several steps:

- Verify customer's credit standing
- Check inventory if we can serve the order
- If credit standing and inventory checks succeeded, goods can be shipped and customer can be billed

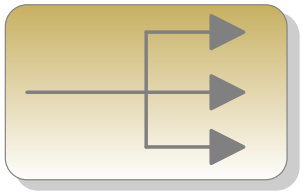
EAI – Patterns – Messaging Example

Processing Orders

- A fork action is required, since two independent systems need to check the incoming order
- A Publish-Subscribe Channel can be used to implement a fork action



EAI – Patterns – Publish-Subscribe Channel



Context: application uses messaging to announce events

Problem: How can the sender broadcast an event to all interested consumers

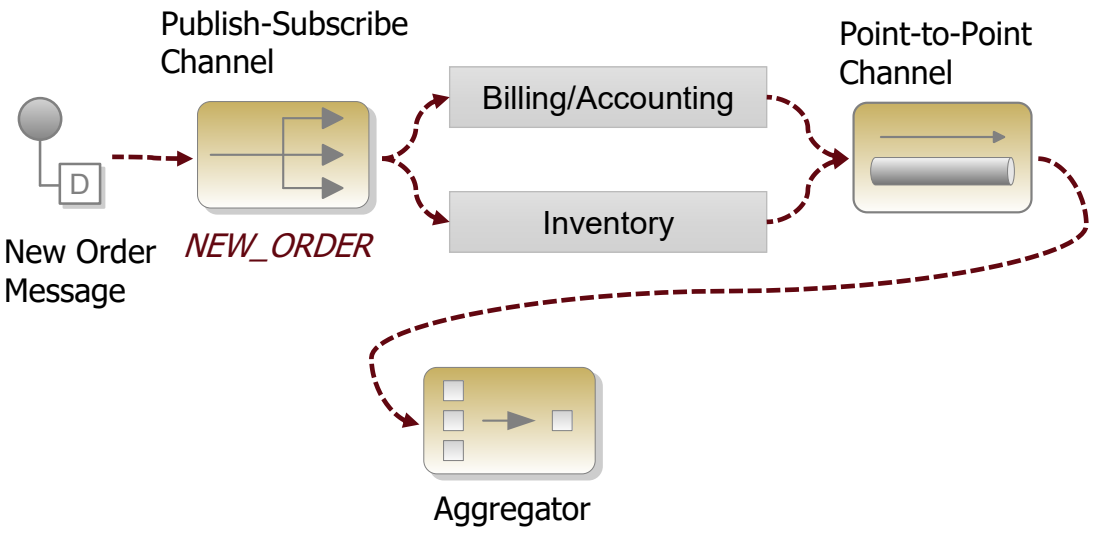
Solution: Publish-Subscribe Channel delivers a copy of a particular event to each consumer (e.g. JMS Topic)

- Internally, a Publish-Subscribe Channel has one input channel and one output channel for each consumer
- Can be powerful for monitoring and debugging purpose → allows to eavesdrop messages (of course this is a danger as well)

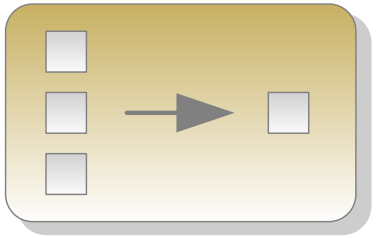
EAI – Patterns – Messaging Example

Processing Orders: A fork action usually requires a subsequent join action

- Using Aggregator to verify that both checks succeeded (or not) – i.e. to aggregate result from the independent systems



EAI – Patterns – Aggregator



Context: multiple sub messages must be considered to make a decision

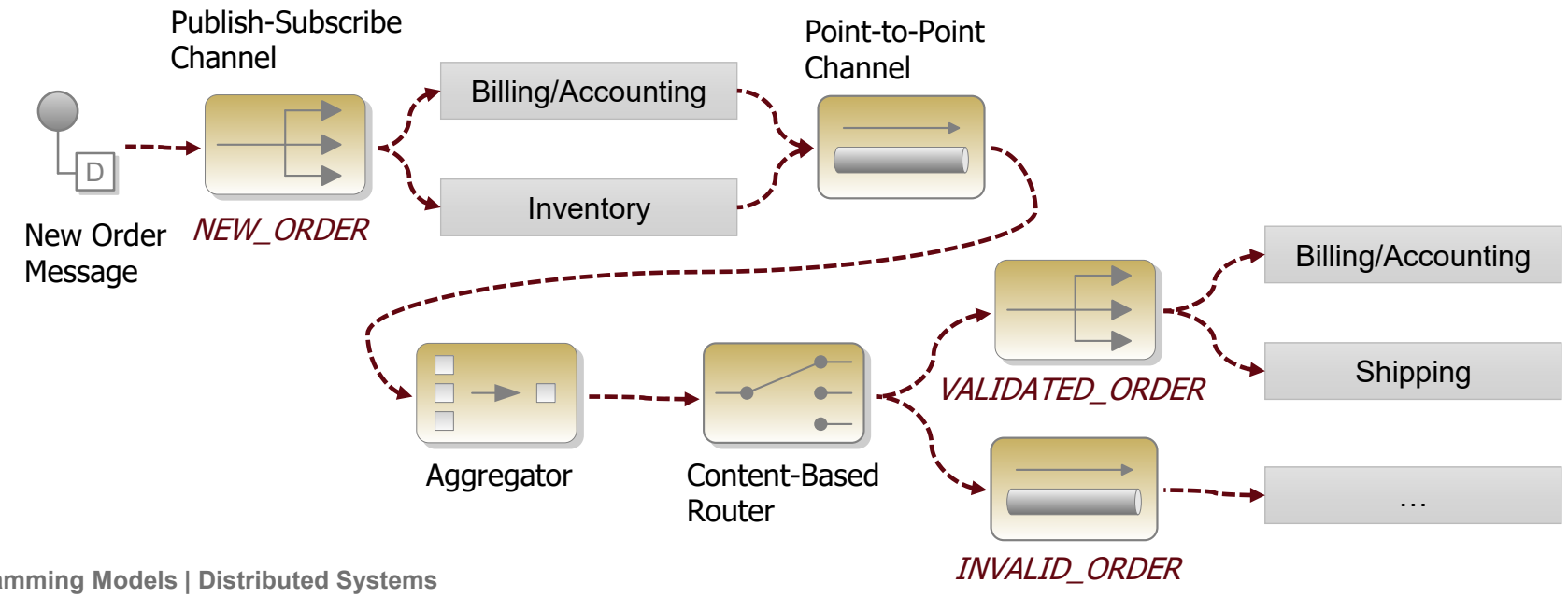
Problem: How to combine the results of individual but related messages?

Solution: Aggregator collects individual messages until a full set of related messages is received. Aggregator publishes a single message compiled from related inputs

- Aggregators are stateful components, i.e. implementations need to cope with:
  - Correlation – which incoming messages belong together: Cannot use customer ID (in case of multiple orders) – must introduce correlation ID (e.g. using Data Enricher)
  - Completeness - when are we ready to publish the result: Simple: we require two messages
  - Aggregation Algorithm – how to combined the single messages into a result: Simple as well: just compute overall result, boolean result = billingResult && inventoryResult

EAI – Patterns – Messaging Example

Processing Orders: After having aggregated the result → decision whether the order is valid or invalid → Content-Based Router and two channels for valid and invalid orders



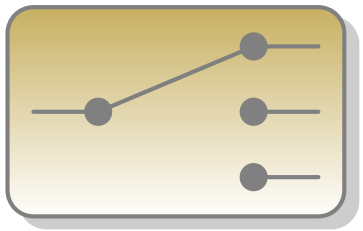
EAI – Patterns – Content-Based Router

Context: many enterprise integrations scenarios must deal with several systems performing same functions – e.g. multiple inventories

Problem: How to handle a situation in which the implementation of a single logical function is split across multiple systems?

Solution:

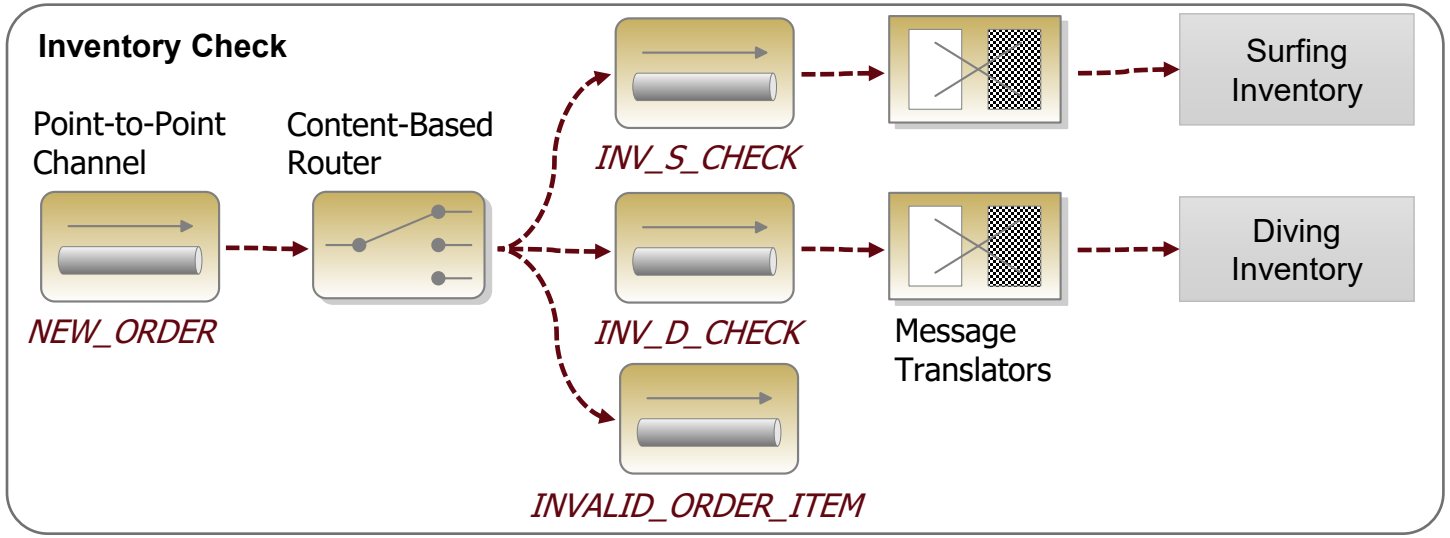
- Content-Based Router routes each message to the correct consumer based on the content of the message
- Content-Based Router checks the existence of fields or evaluates field contents to make decisions. The router incorporates knowledge of other systems → changes to other systems affect the router
- The router can be implemented as a configurable rule engine



EAI – Patterns – Messaging Example

Processing Orders: The inventory systems needs treatment, because we have two of them!

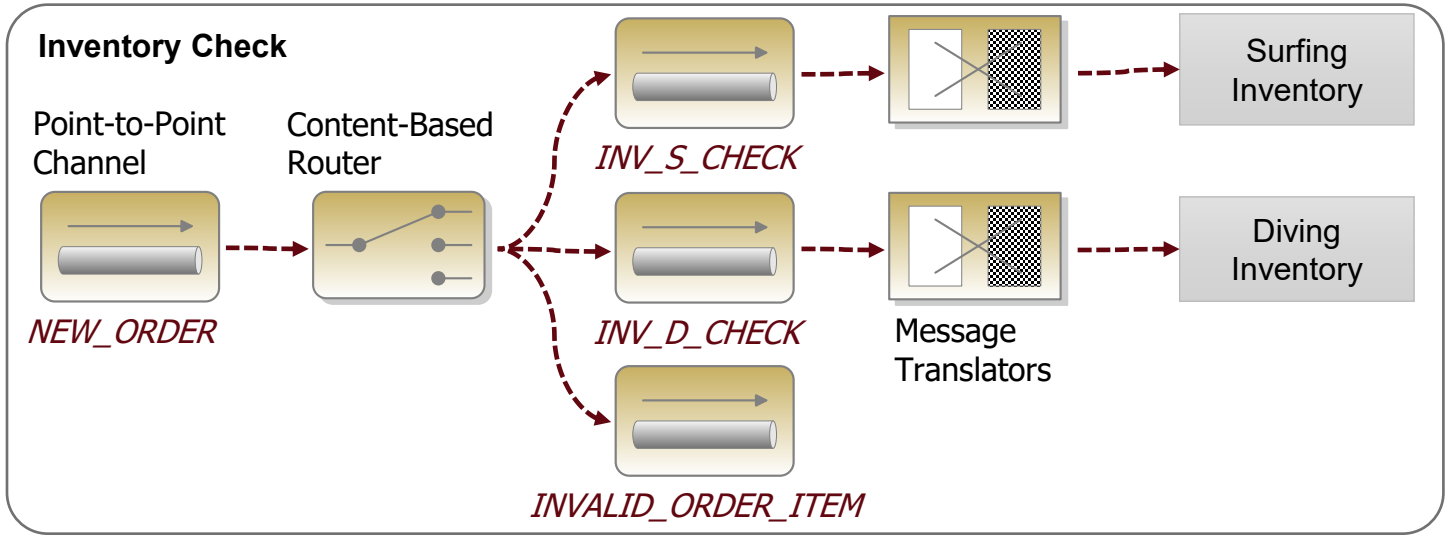
- To encapsulate this from the rest of our system → insert another Content-Based Router
- Message Translators to comply to the format used by the inventories



EAI – Patterns – Messaging Example

Processing Orders: If order does not match an inventory, it is routed to invalid message channel

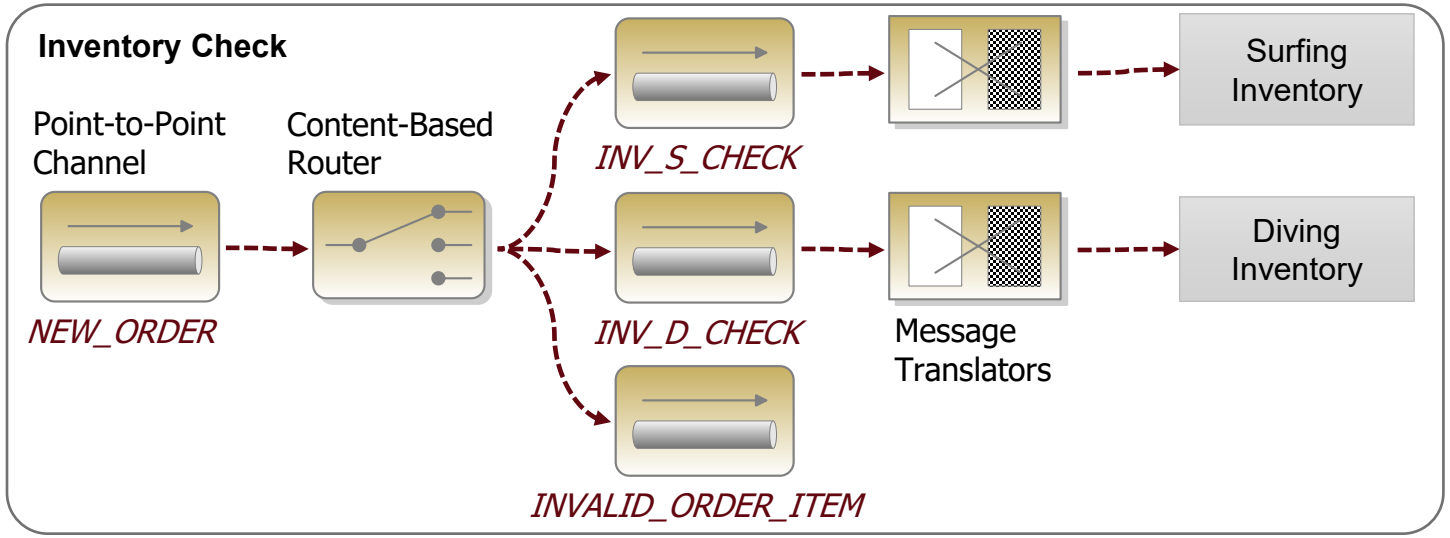
- This highlights that depending on the channel, messages become different meaning (all channels transport the same type of messages)



EAI – Patterns – Messaging Example

Processing Orders: What happens with orders that contain multiple items (possibly ranging multiple inventories)?

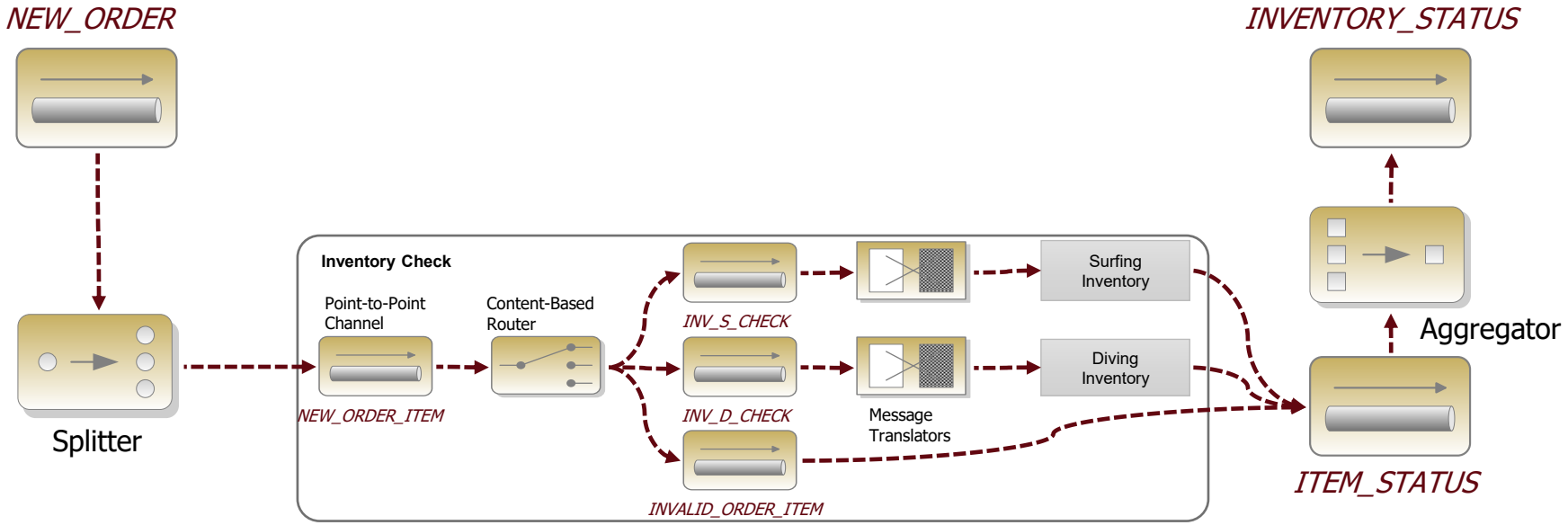
- Instead of a Content-Based Router → use Publish-Subscribe Channel
- What happens with invalid orders? → insert a Splitter and an Aggregator



EAI – Patterns – Messaging Example

Processing Orders

- Splitter breaks single order message into multiple order item messages
- Aggregator recombines the single results



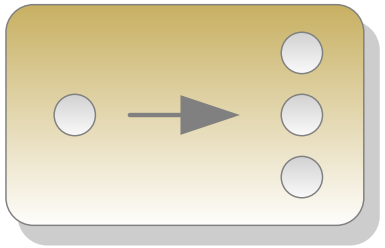
EAI – Patterns – Splitter

Context: many messages passing through an integration solution consist of multiple elements

Problem: How to process a message that contains multiple elements, each of which may have to be processed individually?

Solution:

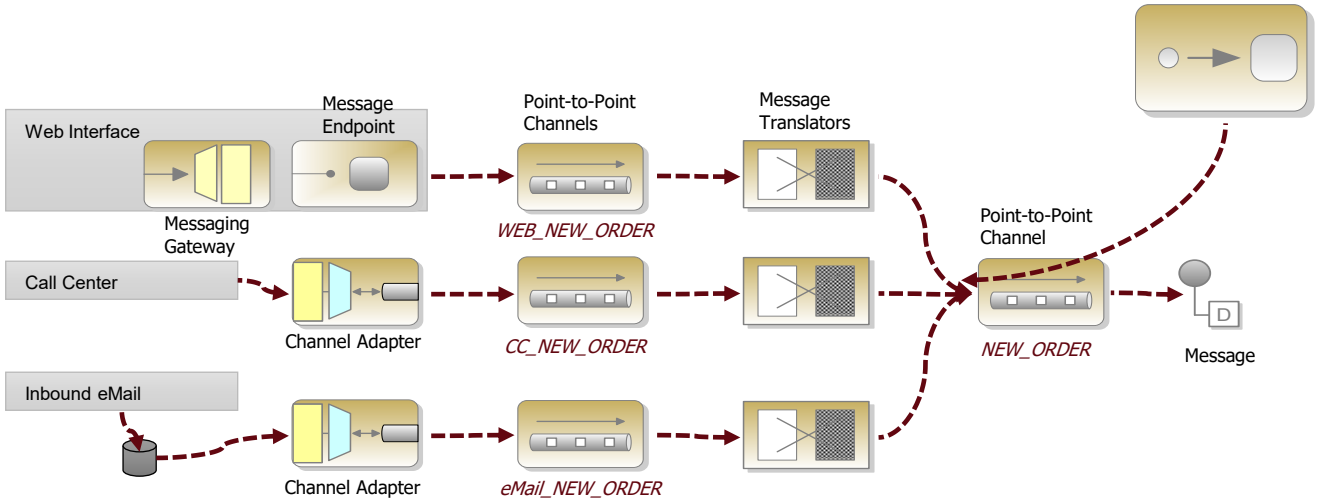
- A Splitter splits a composite message into a series of individual messages, each containing data related to one item
- To make the resulting message self-contained, some common elements are repeated in each result message (e.g. correlation ID, number of items in order) → Resulting messages may also contain a reference to the original one
- Sometimes splitter introduce sequence numbers (message order)



EAI – Patterns – Messaging Example

Processing Orders: where to get the Correlation ID from?

- Content Enricher for taking order integration



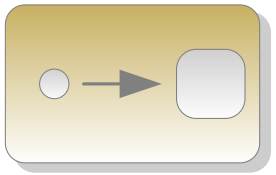
EAI – Patterns – Content Enricher

Context: in enterprise integration, it is common that the source system lacks information required by the target system

Problem: How to communicate with another system if required data is missing?

Solution: Content Enricher is a specialized transformer

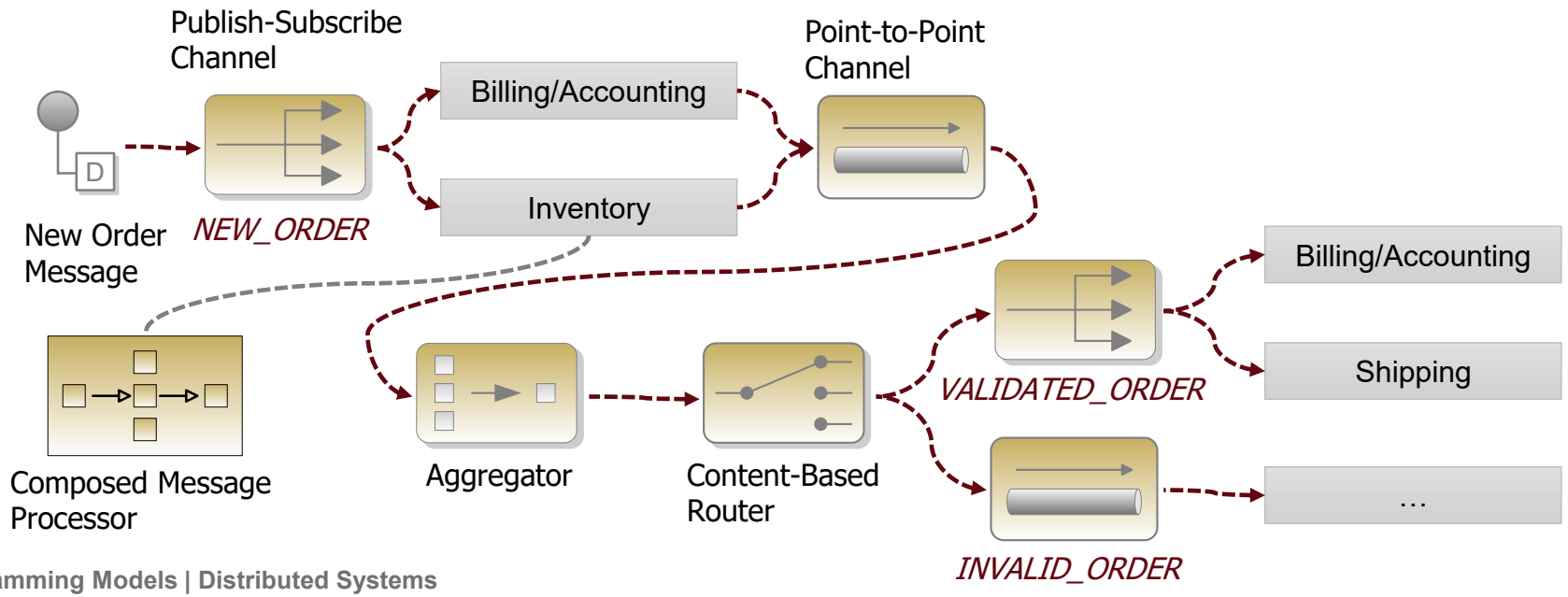
- It accesses an external data source to augment the message with missing information
- Common sources for missing information:
  - Computation: the enricher can compute the data (e.g. the Correlation ID)
  - Environment: the enricher can retrieve the information from the environment (e.g. a timestamp)
  - Another system: the enricher retrieves data from another system (e.g. a file or an LDAP directory)



EAI – Patterns – Messaging Example

Processing Orders: Combination of a Splitter, Aggregator and Message Router is common

– To simplify our picture, we can substitute these elements by a Composed Message Processor



EAI – Patterns – Summary

Enterprise Application Integration based on messaging is a complex task

- Various components for selection, routing, transformation, aggregation, … are required
- The given example is simplified, real world scenarios are much more complex
- Patterns and pattern languages help to design complex systems
- They provide an understanding of the composition and relation of integrated applications and help to select appropriate solutions for common integration problems

Questions ?